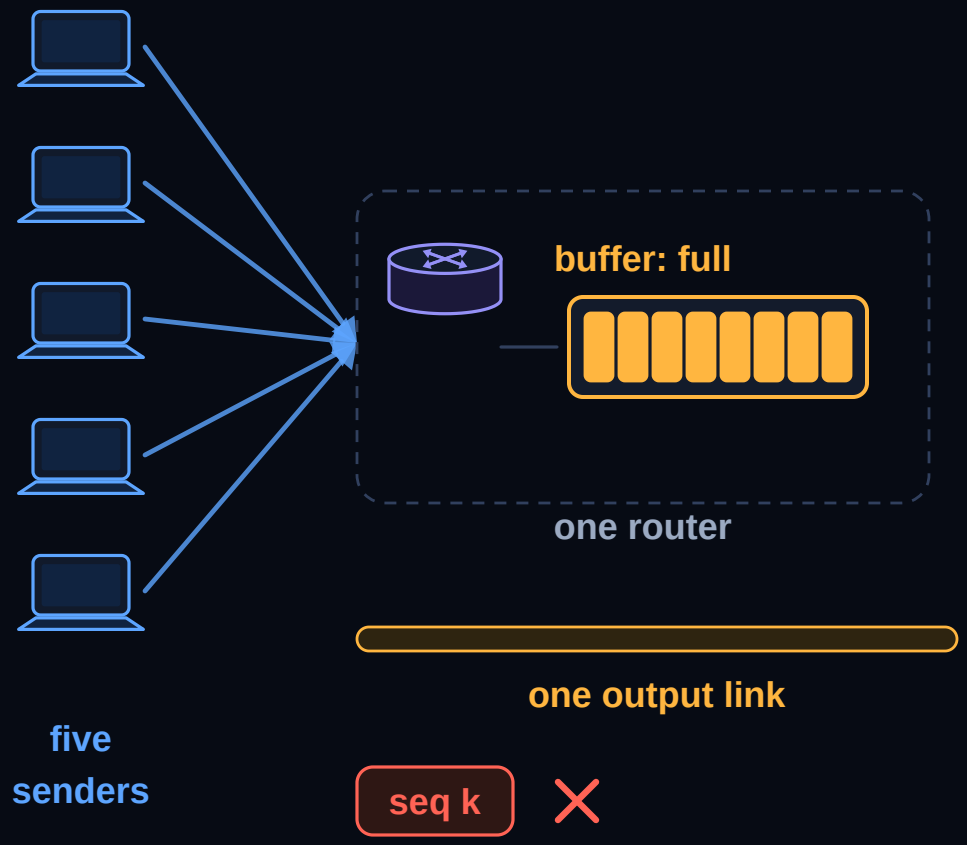
Congestion Control
How fast should a sender send?

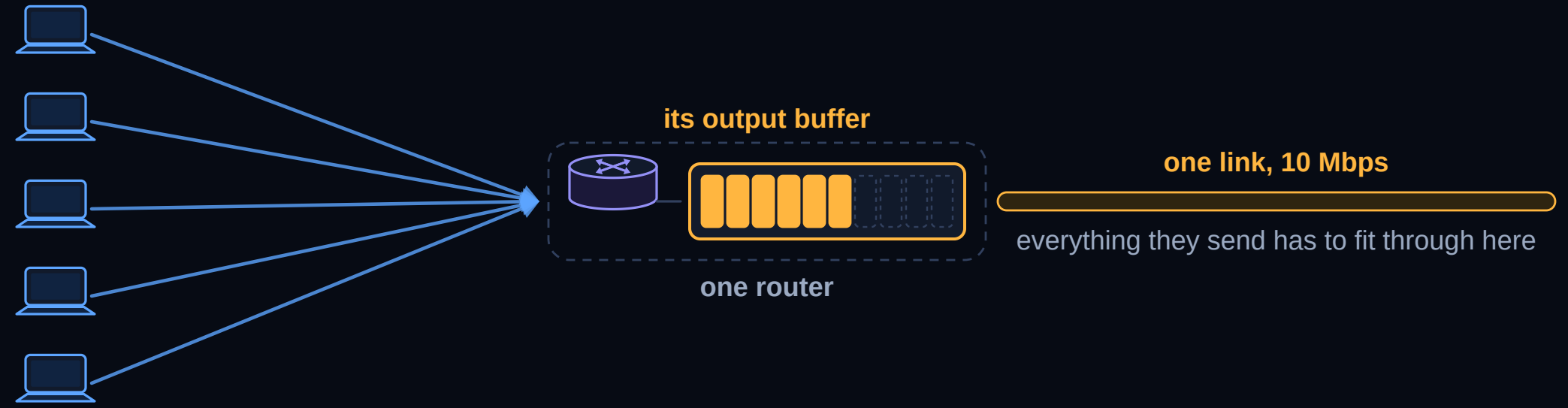
Every link in the internet is shared and nobody hands out the shares. This lecture shows what goes wrong when senders get their rate wrong, how a sender can find out, and the rule that has made every sender limit its own rate since 1988.

Yaxiong Xie



nobody told any of them how fast to send

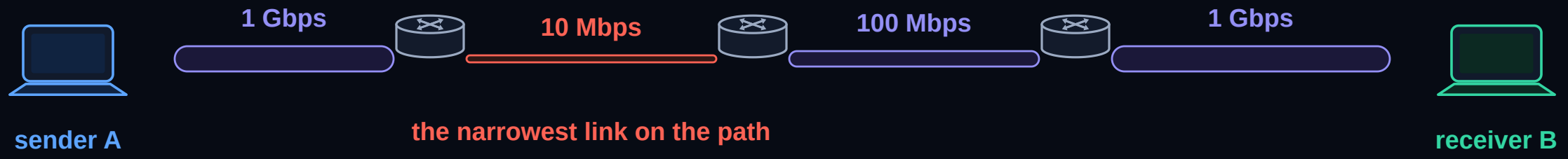
Who decides how much of a link each sender gets?



- no reservation is made
- no admission control
- no router hands out shares

Nobody. A sender may put packets on the internet at any rate, at any moment.

So how fast should a sender send?



✗ **as fast as the application produces data**

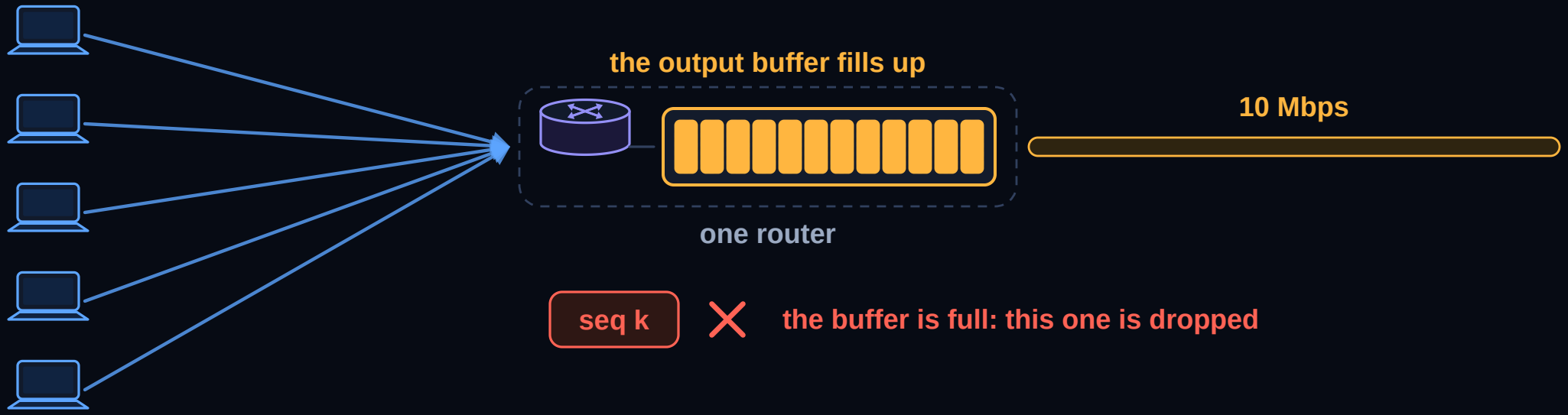
the application has no idea what the path can carry

✗ **as fast as sender A's own card can go: 1 Gbps**

a hundred times what the 10 Mbps link can take

The right rate is set by the slowest link and by who else uses it. A sees neither.

What happens when senders get it wrong?



First it delays

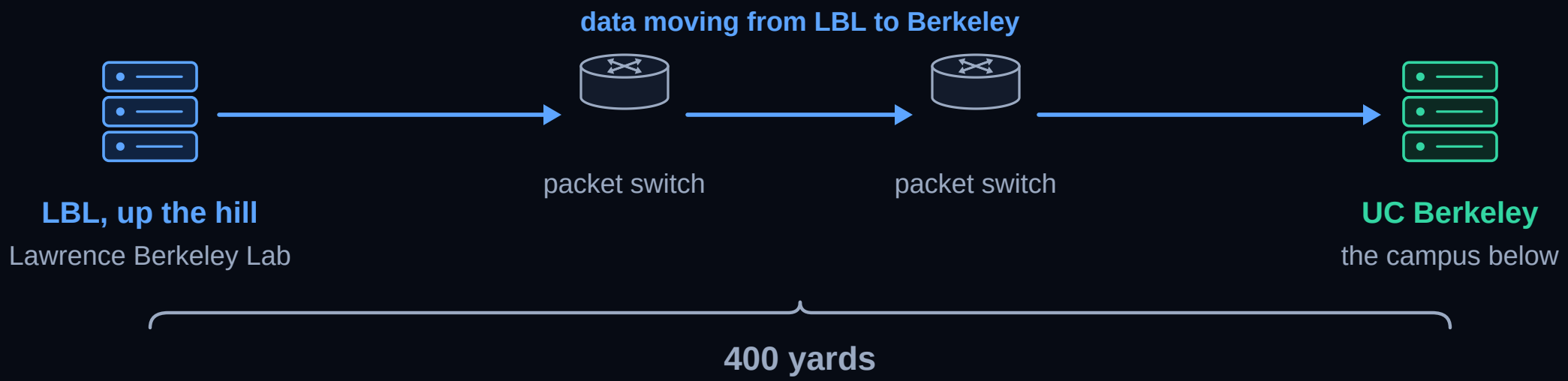
a packet that fits waits behind everything already in the buffer

Then it throws away

when the buffer is full there is nowhere left to put a packet

The network has two answers to too much traffic: delay it, then throw it away.

October 1986: two sites, 400 yards apart, two hops



- ✓ Every link was up
- ✓ Both switches were up
- ✓ No hardware had failed

Van Jacobson called it the first of what became a series of “congestion collapses” (SIGCOMM 1988).

Every link and every packet switch was working exactly as it was designed to.

The same path, 800 times slower

THE THROUGHPUT FROM LBL TO UC BERKELEY

normally

32 kbps

= 4,000 bytes per second

a 1 MB file would take

just over 4 minutes



October 1986

40 bps

= 5 bytes per second

a 1 MB file would take

2.3 days

800 times slower, with the same hardware on the same path

This is what a network does when every sender sends as fast as it can.

Why did the throughput fall instead of levelling off?



rises
more in, more out

flattens
the link is full

falls
more in, less out

Rising and flattening are not surprising: a link has a capacity. Falling is. Part 4 takes the falling part apart.

Past capacity, every extra packet offered reduces what the network delivers.

What did the internet fix after 1986?

✕

No reservation protocol

was ever deployed on
the public internet

✕

No router was redesigned

a full buffer still means
a packet is dropped

✕

Nobody in charge

no authority hands out
any sender's share

The 1986 collapse could happen again tomorrow.

Every condition that produced it is still in place: shared links, no reservations, no admission control, and buffers that still overflow.

None of it. The public internet still reserves nothing and refuses no one.

Why doesn’t that happen every day?

What changed in 1988 was only this:

every sender limits its own rate,
and keeps adjusting it, forever

no message from the network, no coordination between senders

in the hosts, not the network

one algorithm, added to TCP
in every operating system

nothing had to be agreed

each host could adopt it on
its own, one machine at a time

and the idea still runs

on the machine in front of you,
though the exact rule has changed

Congestion control is a rule the end hosts keep. This lecture is that rule.

Three questions, eight parts

01



What exactly goes wrong

inside the network when senders send too much?

1 Congestion
2 Causes
3 Challenges
4 The costs

02



How can a sender control

its rate, and find the right one when nothing tells it?

5 The rate
6 Probing

03



How should the rate move

up and down, so that senders share fairly, and how does a connection start?

7 Rate control
8 Slow start

Nothing in this lecture needs a change to any router. It is all in the sender.

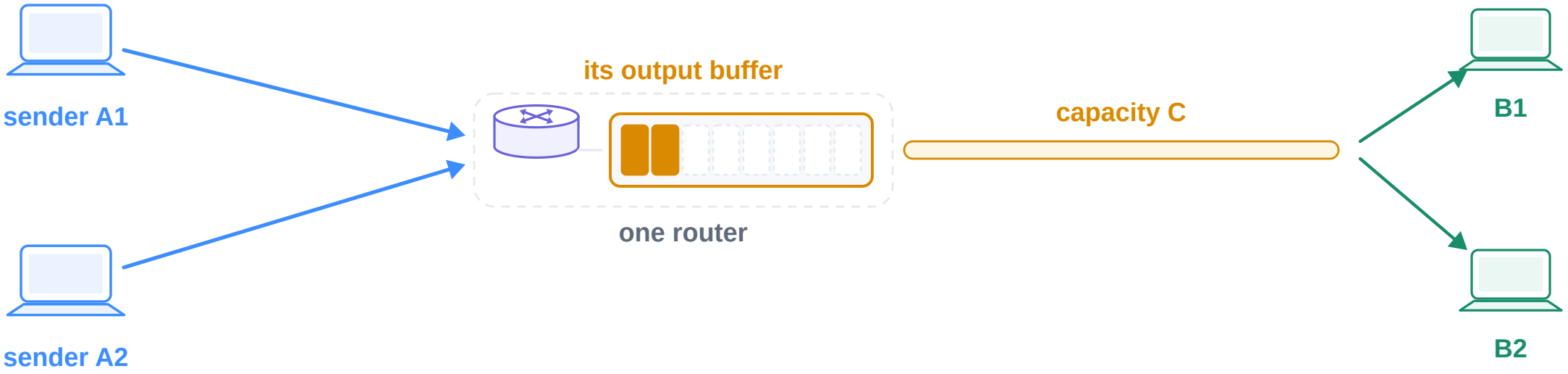
What is congestion?

01

What congestion is

Two senders share one link. When together they send more than it can carry, the router’s buffer takes the difference, fills up, and then drops packets. This part watches that happen, one step at a time, and then names it.

The model: two senders, one router, one output link



They never meet

A1 cannot measure A2, and neither of them can see the buffer

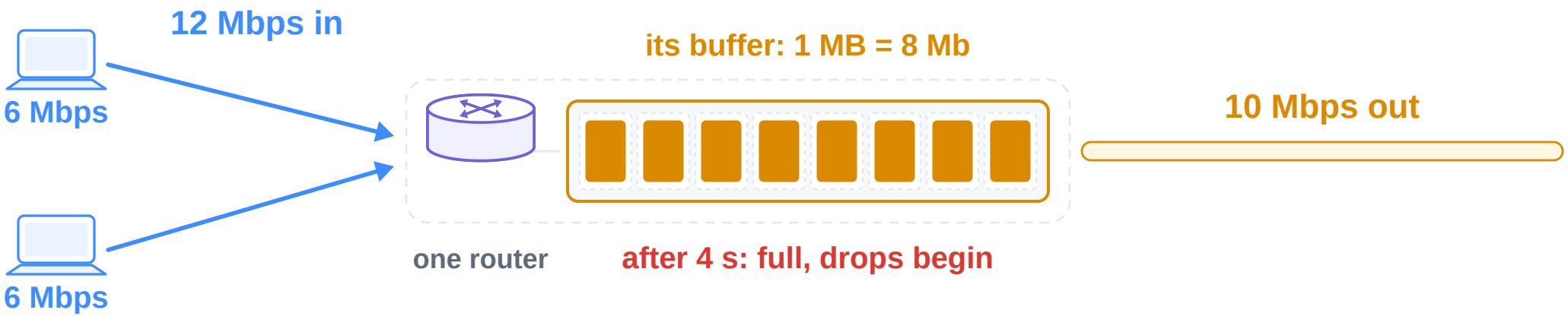
They share one link

every packet either of them sends goes through the same buffer

The smallest picture in which congestion can happen at all.

Nothing in this picture says how fast either sender is allowed to send.

12 Mbps in, 10 Mbps out. Where does the rest go?



12 Mbps arrive, but only 10 Mbps can leave.

every second: 12 Mb in and 10 Mb out, so 2 Mb stay in the buffer, until it is full

For 4 s the buffer absorbs it

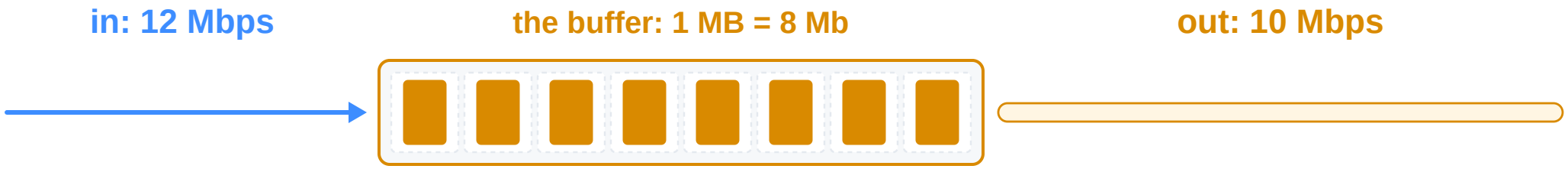
nothing is lost yet, but waits keep growing

Then it is full

the 2 Mb a second the link cannot send are dropped

Into the buffer, 2 Mb more each second, until it is full. Then the extra is dropped.

How long does 2 Mbps take to fill a 1 MB buffer?



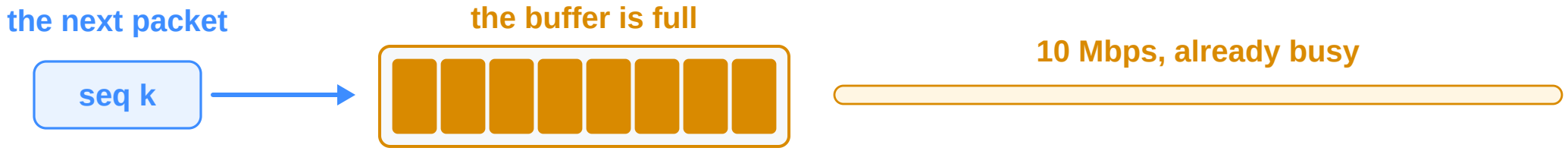
after 4 s: 8 Mb waiting, the buffer is full

fill rate	**= in − out = 12 − 10 = 2 Mbps**
fill time	**= size ÷ fill rate = 8 Mb ÷ 2 Mbps = 4 s**

An 8 MB buffer (64 Mb) instead: 64 ÷ 2 = 32 s. The same end, 28 seconds later.

Four seconds. A bigger buffer changes when that happens, not whether it happens.

The buffer is full. What could the router do?



Hold it for later?

every slot is taken, and the router has no other memory

✕ no memory left

Slow the sender down?

IP has no field to say it with; Source Quench tried, and failed

✕ no way to say it

Send it out faster?

the link sends at 10 Mbps, and that is the link, not the router

✕ no faster link

A router has no memory to spare, no field to speak through, and no say over the link rate.

So the router throws the packet away



It discards the packet.

a tail drop: what every router does by default when a buffer is full

And says nothing about it.

no message goes back to the sender; only its own drop counter changes

The sender finds out only because an ACK never comes.

In classic congestion control that silence is the only signal a sender gets, and Part 6 is built on it.

With nowhere to put it and no way to say so, the router drops it in silence.

So what is congestion?

1 More arrives than leaves



a short burst is absorbed, and the buffer drains: not congestion

2 It goes on: congestion



the queue keeps growing and does not drain: waits get longer

3 Full: packets are dropped



4 s later the buffer is full: new arrivals are dropped, silently

Congestion: more arrives at a link than it can send, for long enough that a queue builds and does not drain.

Delay grows first; once the buffer is full, packets are dropped.

Nothing is broken, just as in 1986: every link and router is doing its job. There is simply more traffic than the link.

Congestion is a queue that does not drain: delay comes first, drops come last.

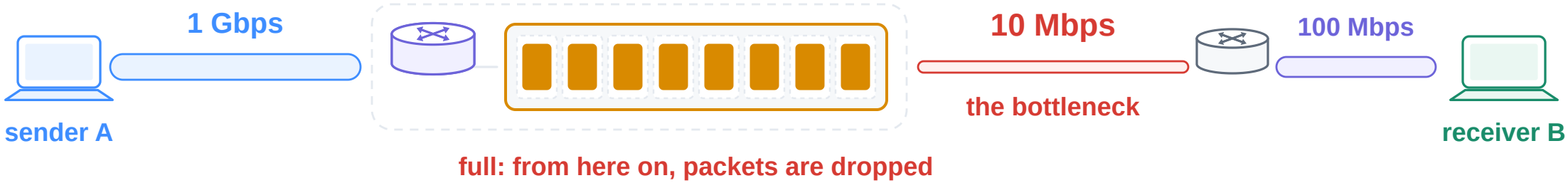
Why does congestion happen?

02

Why congestion happens

Part 1 watched a buffer fill because more arrived than its link could send. This part asks how a network gets there: one sender faster than its path, or many senders that are each slow enough and together too fast.

Can one sender congest a path on its own?



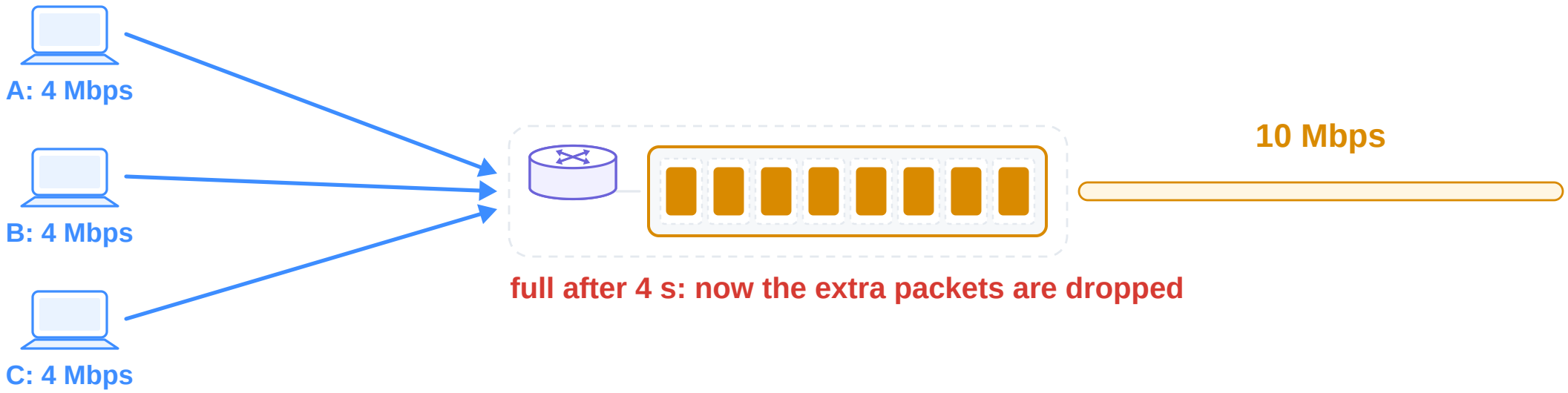
A cannot see the 10 Mbps link. Its own card runs at 1 Gbps, so that is how fast it sends.

fill time = 8 Mb ÷ (1,000 − 10) Mbps ≈ 8 ms

slide 14’s arithmetic: a 1 MB buffer lasts eight milliseconds

Yes. A sender that cannot see the bottleneck overruns it on its own, in milliseconds.

Can senders that each stay below the link still congest it?



✓ **each: 4 Mbps, well below 10**

no sender does anything wrong

✗ **together: 4 + 4 + 4 = 12 Mbps**

Part 1's two senders were this case: 6 + 6

Yes. Every sender can be well below the link, and all of them together still overrun it.

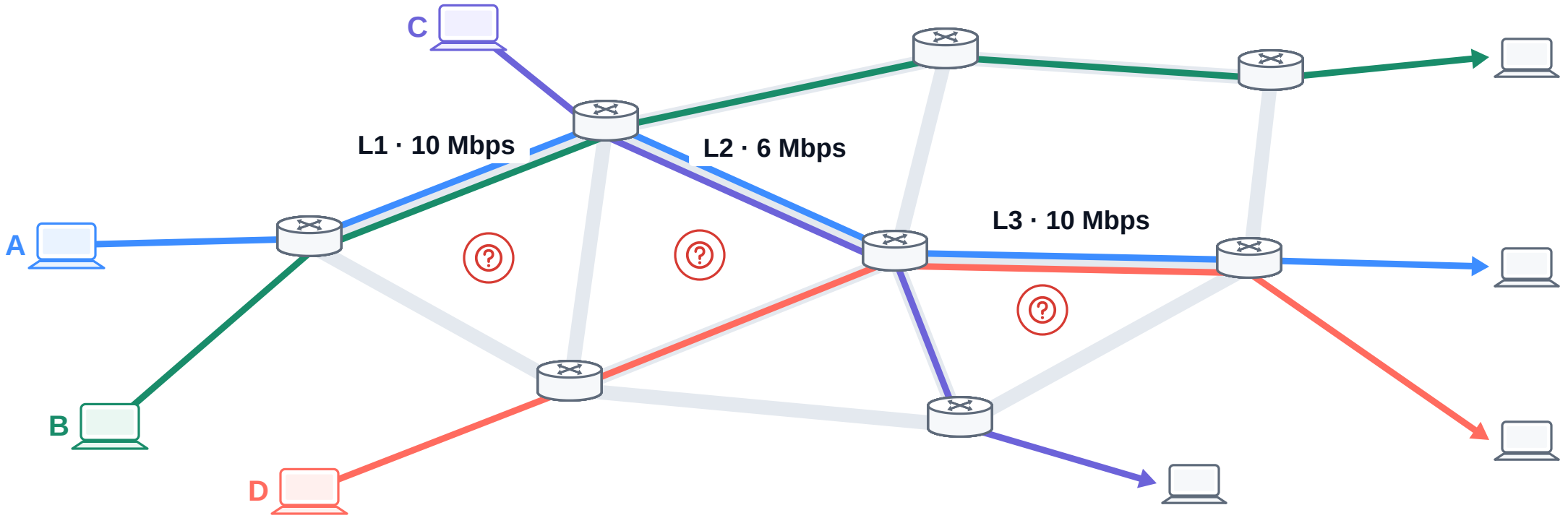
Why is congestion control challenging?

03

Why control is hard

Part 2 showed how senders overrun a link. Stopping that is harder than it sounds: the internet is a mesh, every flow shares links with flows it cannot see, the senders cannot coordinate, and even a rate that is right now does not stay right.

The internet is a mesh: who shares which link?

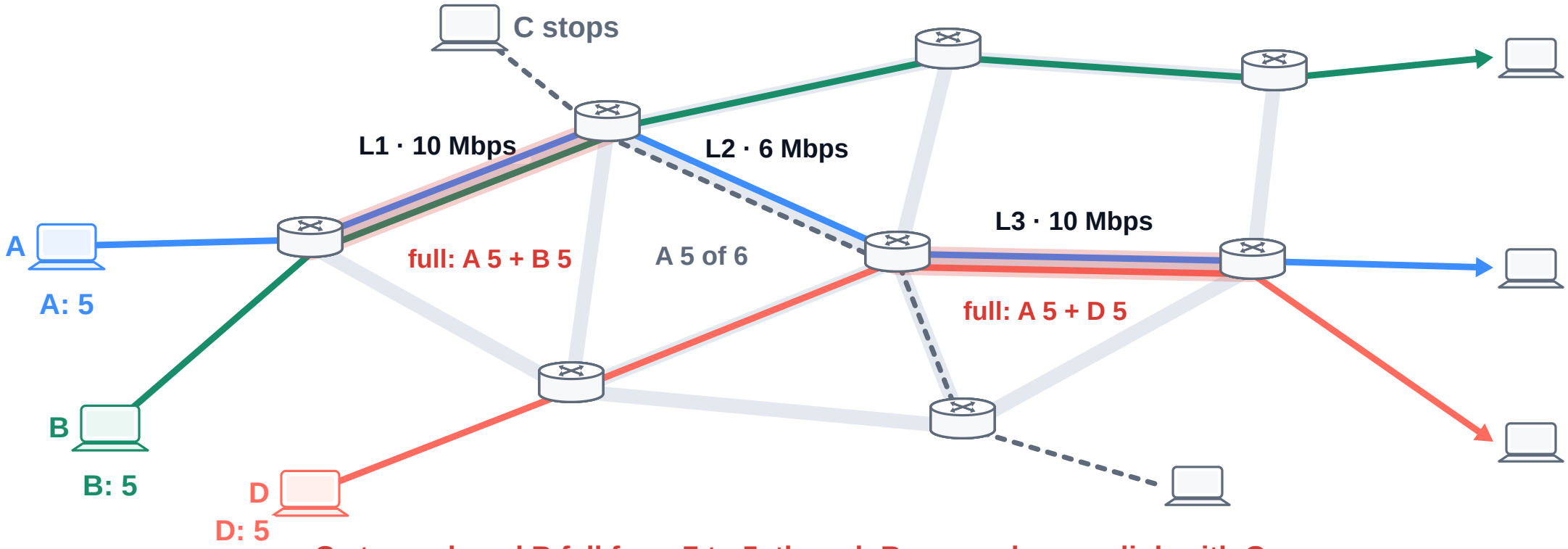


So B, C and D can each slow A down, on three different links.

When A's packets wait or get dropped, A cannot tell which flow is the cause.

A shares links with B, C and D: any of them can slow A down, and A cannot tell which.

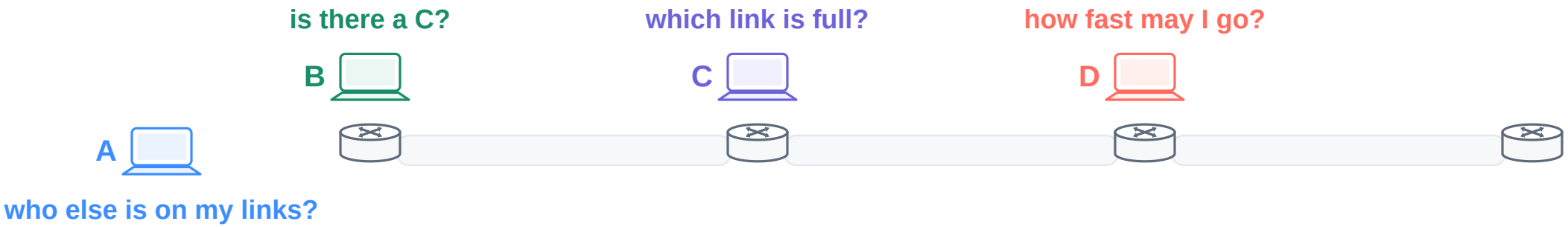
How can a flow that B never meets change B’s share?



C stopped, and B fell from 7 to 5, though B never shares a link with C.

Through A: flows are chained by shared links, so one change moves every share.

Could the senders simply agree on a split?



They cannot see

none knows who shares which link, or which link is full

They cannot talk

nothing carries messages from one sender to another

Nobody is in charge

no router and no authority hands out shares (slide 2)

Agreeing would take a view of the whole network and a way to talk. There is neither.

No. Each sender has to find its rate alone, from what its own packets and ACKs tell it.

Once every sender has the right rate, are we done?

Competition

other flows start and stop,
so each fair share moves
(next slide)

Mobility

a phone moves: its link changes
speed, and its path can change
(the slide after)

And more

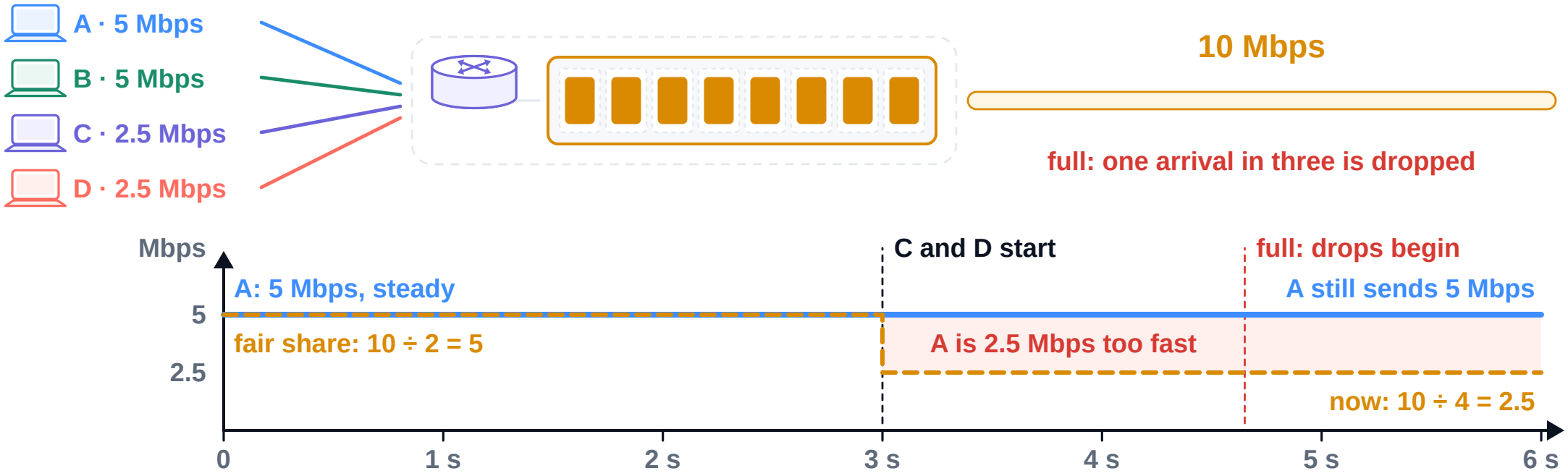
routes change and links fail;
traffic comes in bursts;
each moves the right rate

The right rate is a moving target.

A sender cannot find it once and keep it: it has to keep finding it, alone.

No. Other flows come and go and devices move, so the right rate keeps moving.

Competition: what happens when two more flows start?

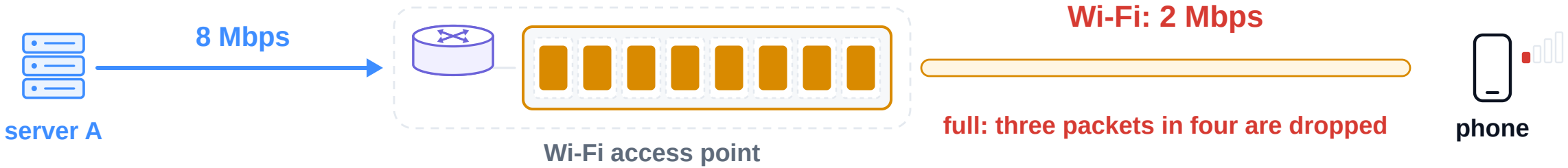


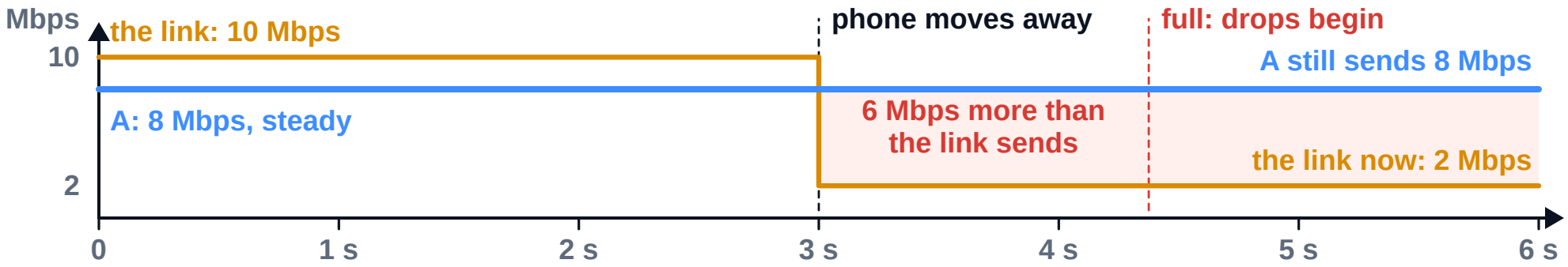
A changed nothing. Its right rate fell from 5 to 2.5 because C and D started.

Nothing tells A that. It finds out only when its packets wait and are dropped.

A's right rate falls when other flows start, and nothing in the network tells A.

Mobility: what if the link itself slows down?





A changed nothing. The link under it fell from 10 to 2 Mbps.

Wireless links change speed all the time: phones move, walls block the signal, other devices share the air.

The right rate moves even with no other flows: the link itself changes speed.

So why is congestion control hard?

A mesh of shared links	each flow shares some links with flows it cannot see
Hidden causes	a sender cannot tell which flow is slowing it down
No coordination	the senders can neither see the network nor talk
Competition	flows start and stop, so every fair share moves
Mobility	devices move, so links change speed

Each sender has to find its rate alone, from its own packets, and keep finding it.

What does congestion cost?

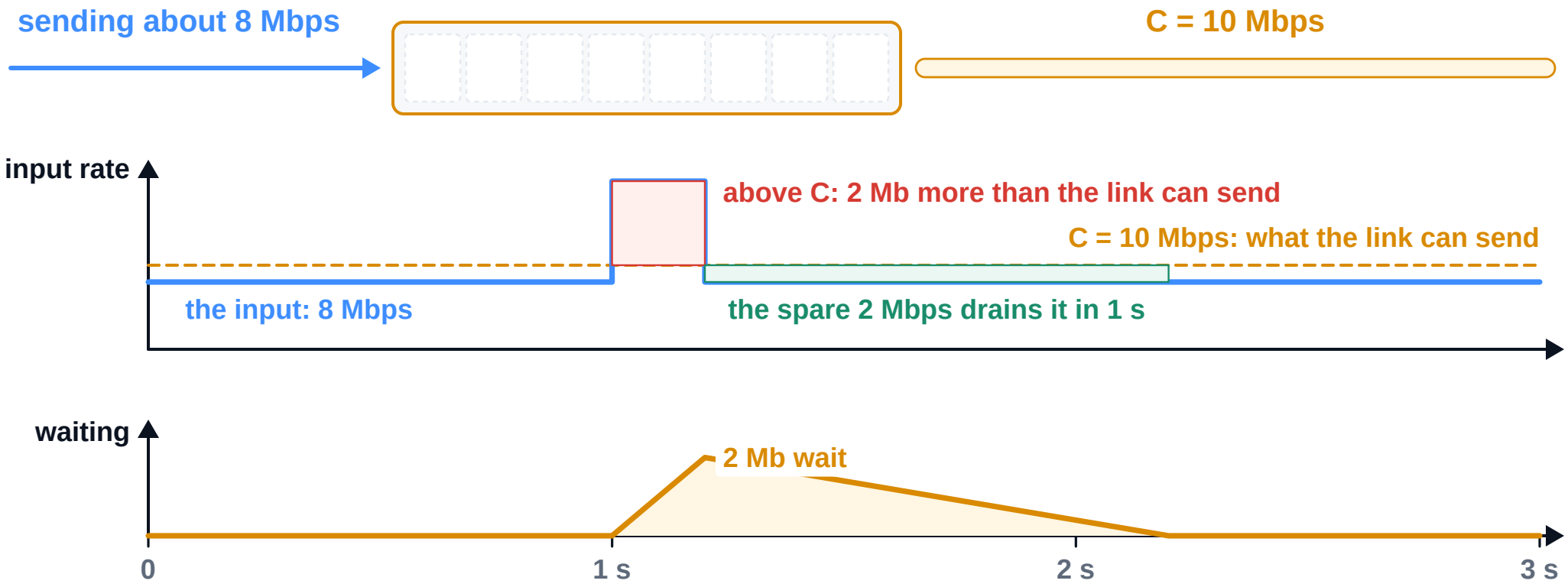
04

What congestion costs

Parts 1 to 3 showed what congestion is, why it happens and why it is hard to control. This part counts what it costs: four costs, each a thing the network spends capacity on and gets nothing back for. Together they explain why the curve on slide 7 falls.

Cost 1: below capacity, why does a packet ever wait?

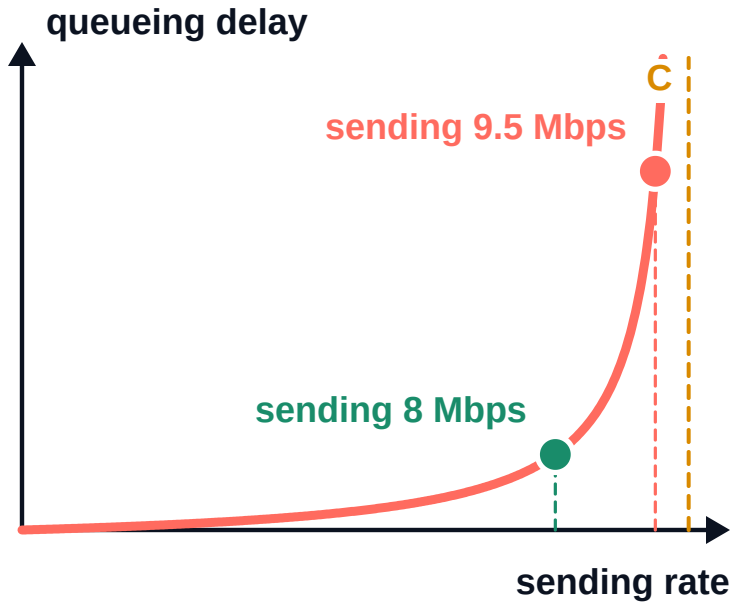
COST 1 OF 4



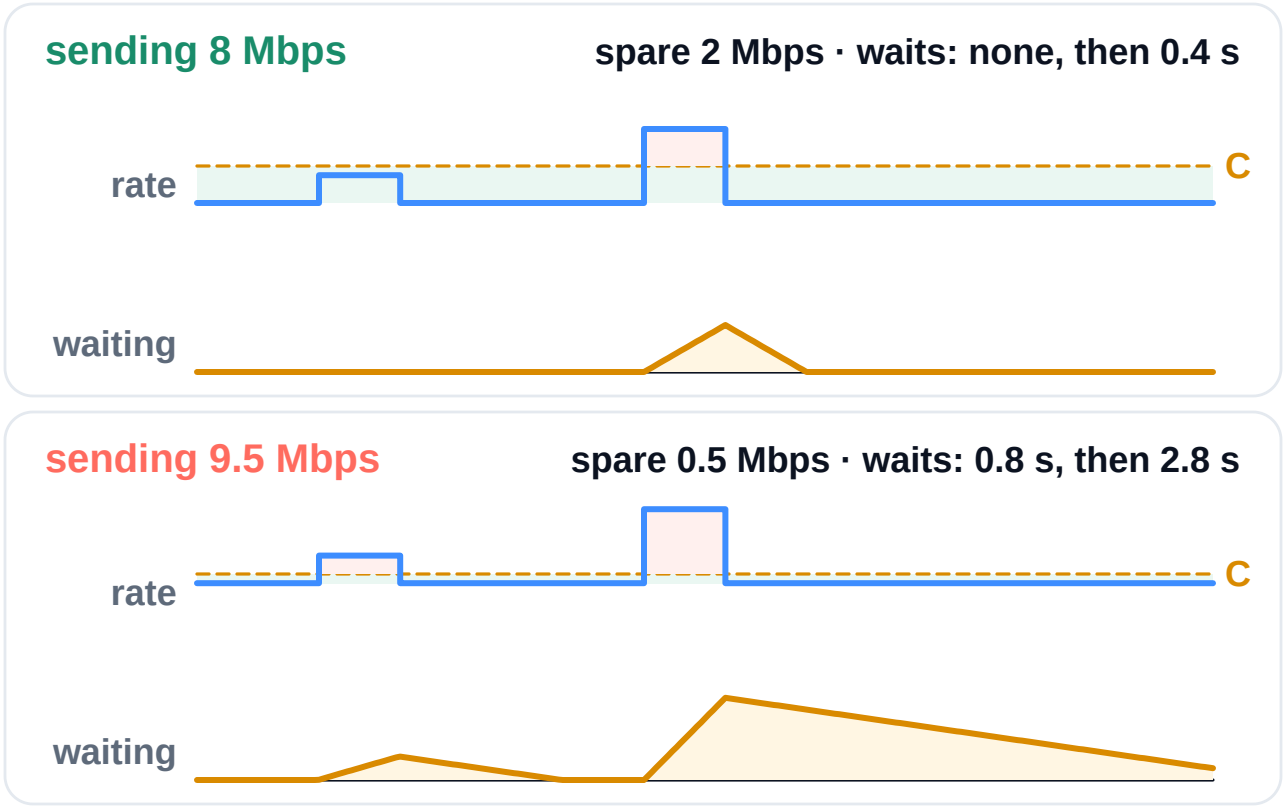
On average the link keeps up; during a burst it does not, and the extra packets wait.

Cost 1: why does the wait grow near capacity?

COST 1 OF 4



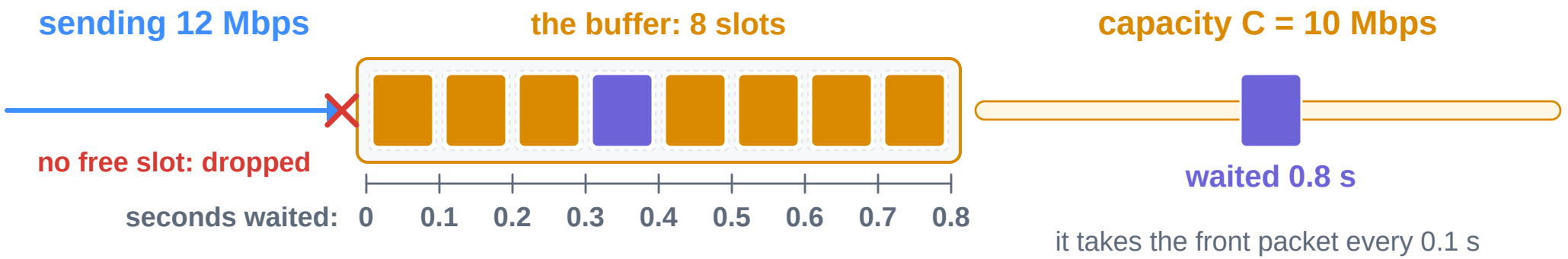
Below C nothing is lost, only delayed.
For a video call, the delay is the cost.

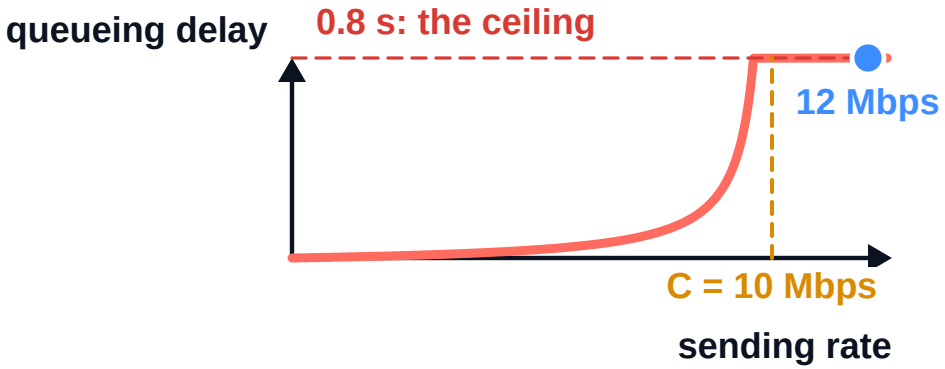


Near capacity a smaller burst starts a queue, and every queue drains more slowly.

Cost 1: once the buffer is full, every packet waits 0.8 s

COST 1 OF 4





Every packet that gets in

waits for the 8 Mb ahead of it: 8 Mb ÷ 10 Mbps = 0.8 s

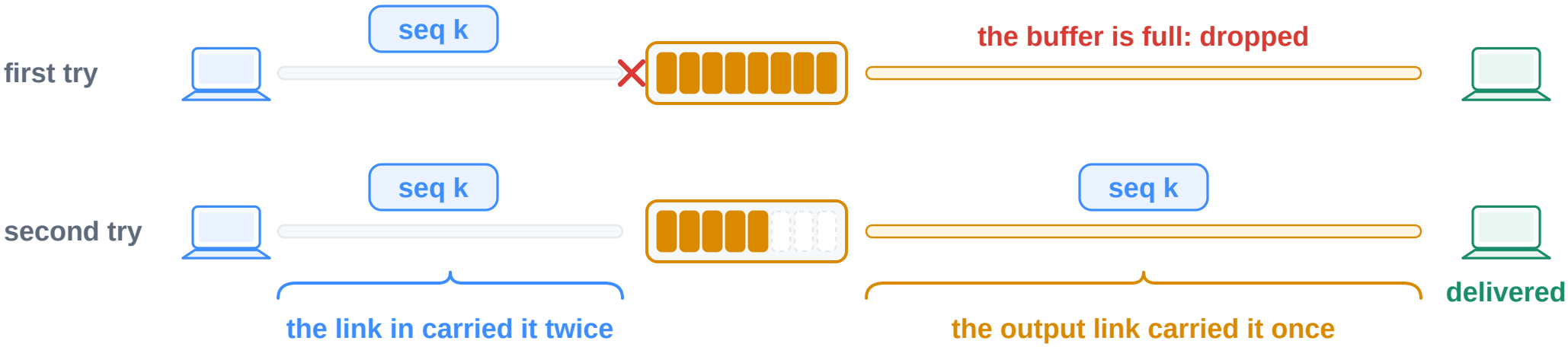
A packet that finds all eight slots taken

is dropped at the door: it waits 0 s, and it is lost

A full buffer makes every packet it takes wait 0.8 s, and drops only those it has no room for.

Cost 2: a dropped packet has to be sent again

COST 2 OF 4

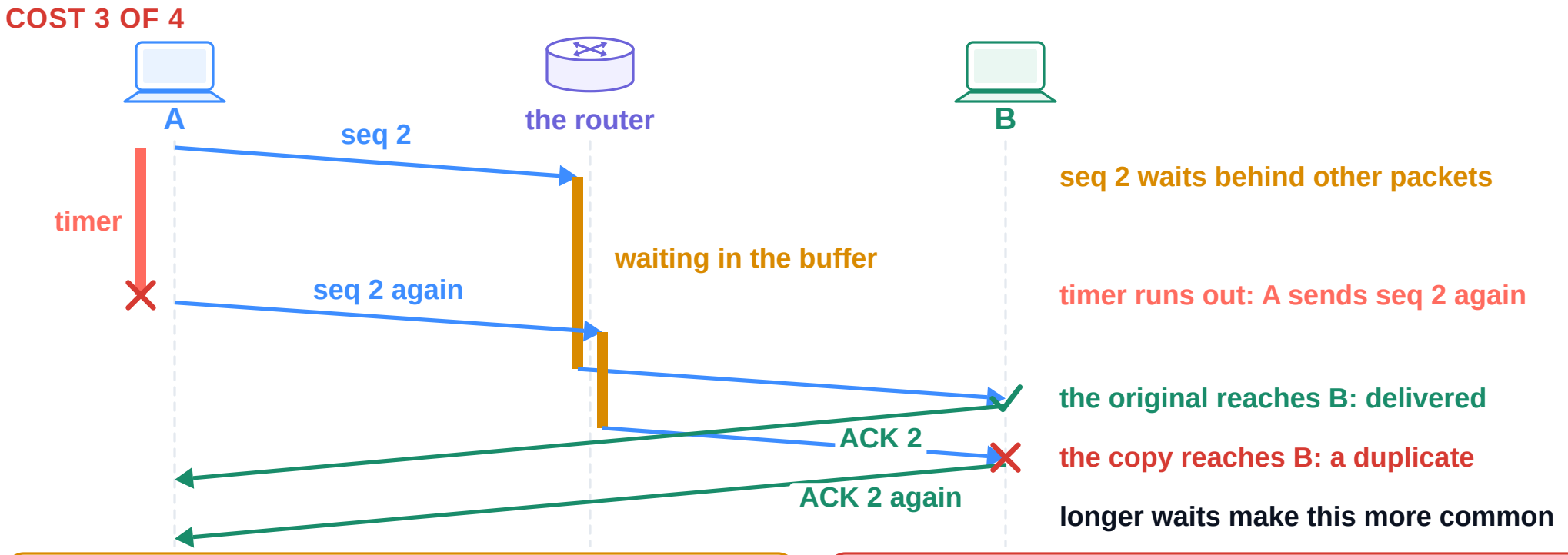


The dropped copy never reached the output link.

The sender and every link before the drop pay for the packet twice.
Resending it takes nothing from the output link: it crosses that link once.

A drop costs every link before it a second copy; the links after it pay once.

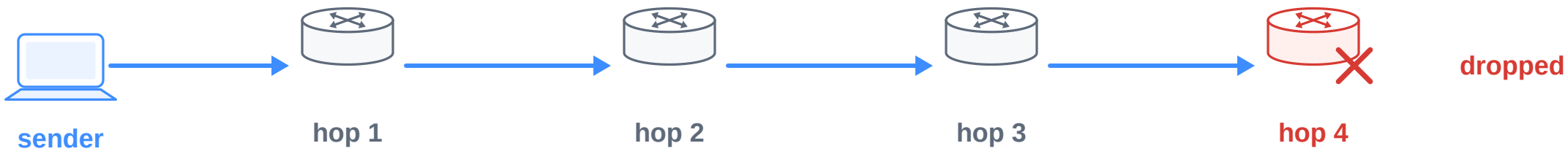
Cost 3: resending a packet that was never lost



Pure waste: the link carries a second copy of data that had already arrived.

Cost 4: a dropped packet wasted every link it already crossed

COST 4 OF 4



four links already carried this packet

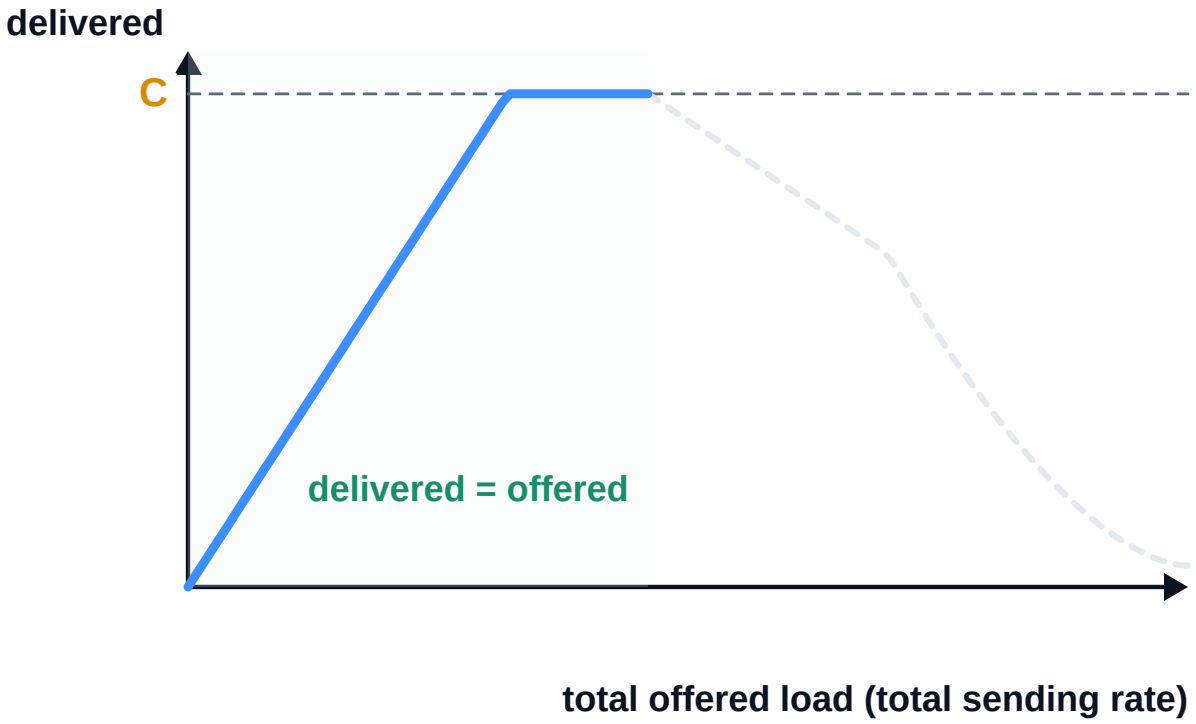
all of that work is now thrown away

Those four links could have carried another connection’s packet, which would have arrived.

Cost 2 was one extra copy on the links before a drop. On a long path those links are shared, so every connection on them pays, and the cost grows with how far the packet got.

The later a packet is dropped, the more of the network’s work it takes with it.

Why does the curve rise, and why does it stop?



✓ Below C it rises

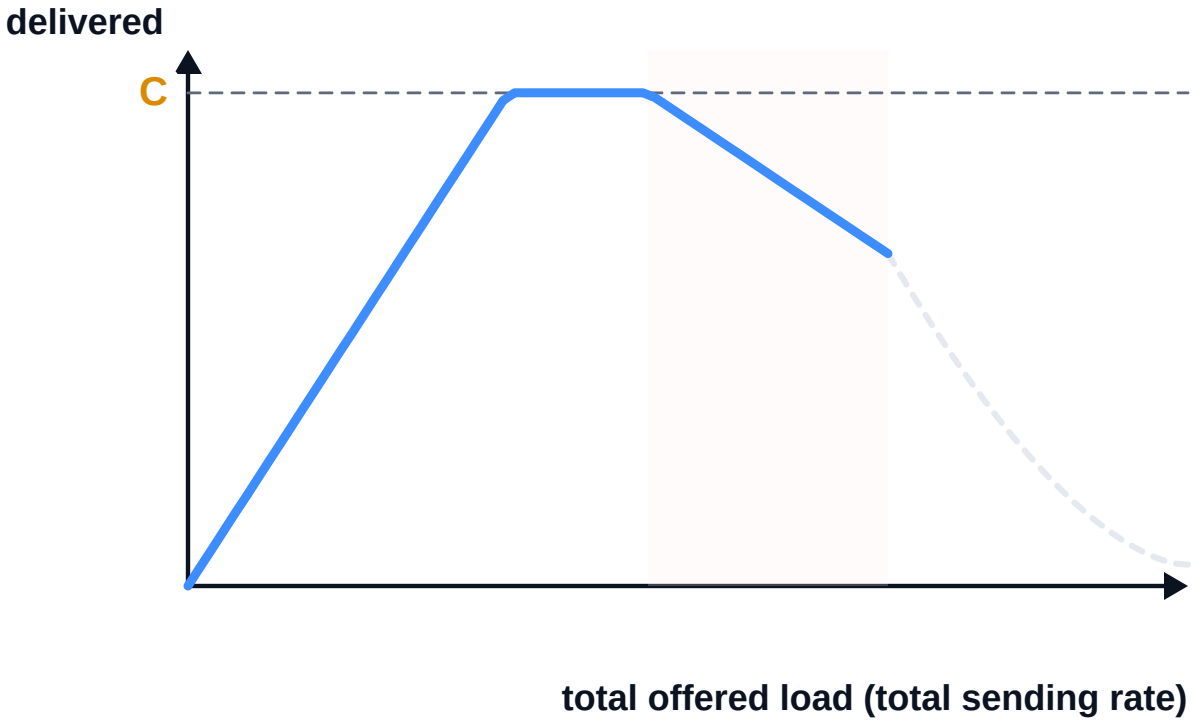
everything offered is delivered: the buffer drains as fast as it fills, so nothing is dropped and nothing has to be sent twice

At C it stops rising

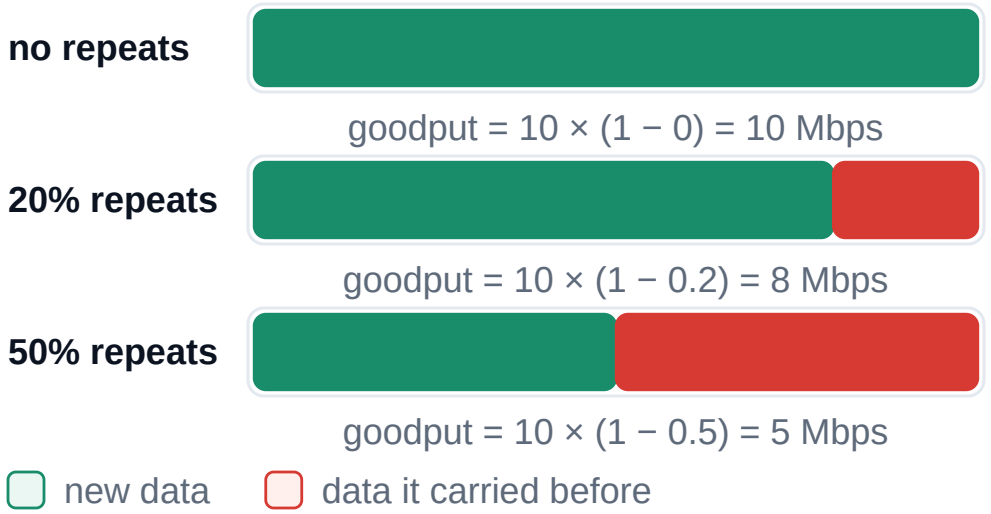
the link transmits C bits every second and not one more, whatever arrives at it.
C is a ceiling on what it can deliver

Up to C, delivered equals offered. C is a hard ceiling on any link.

Why does it fall below the ceiling?



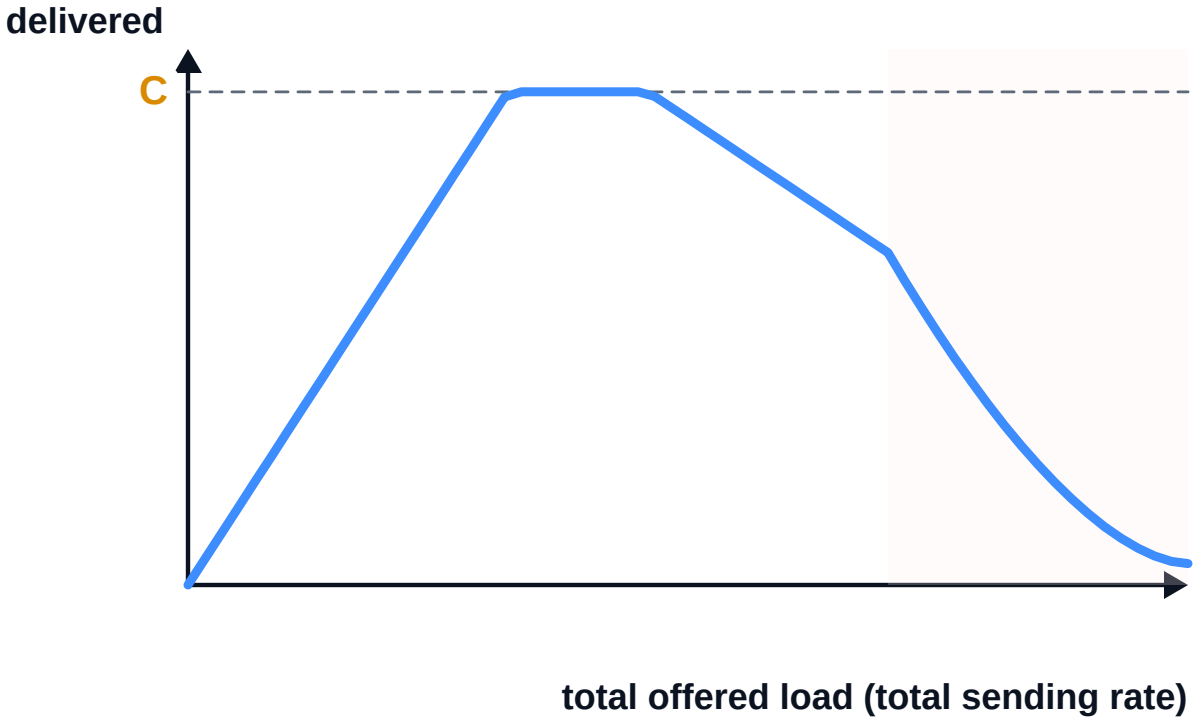
goodput: the green part of the 10 Mbps link



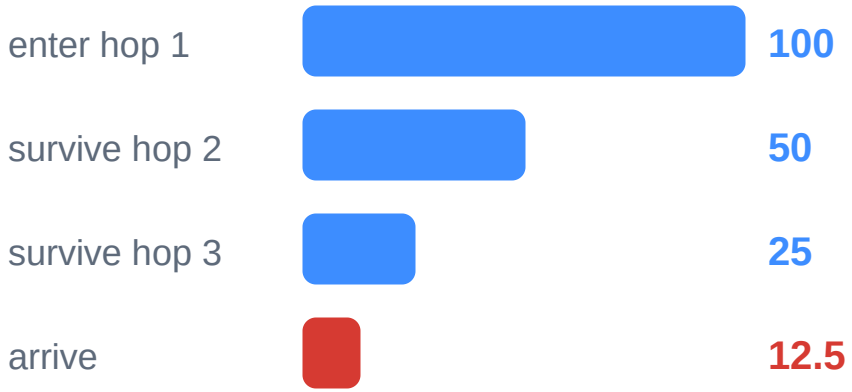
A packet dropped at this buffer never used it.
The repeats are cost 3: a timeout on a packet that was only late, or a window resent whole.

The link is still full. A growing share of what it carries is data it carried before.

Why does it keep falling instead of settling?



four hops; hops 2, 3 and 4 each drop half



Hop 1 did 100 packets of work.
Only 12.5 of them arrived.

The other 87.5 took capacity from packets that would have arrived.

The waste compounds hop by hop, so the delivered share collapses instead of sagging.

Can one sender fix this on its own?

It cannot see the cause

the traffic filling the buffer comes from senders this one has never heard of

It is not the only victim

the packets that get dropped belong to connections that did nothing wrong

Slowing down alone loses

a sender that backs off by itself just hands its share to the ones that did not

So the fix cannot be a rule that one sender keeps.

It has to be a rule that every sender keeps, which is what the rest of the lecture builds.

No. Congestion is the first problem in this course that one sender cannot solve.

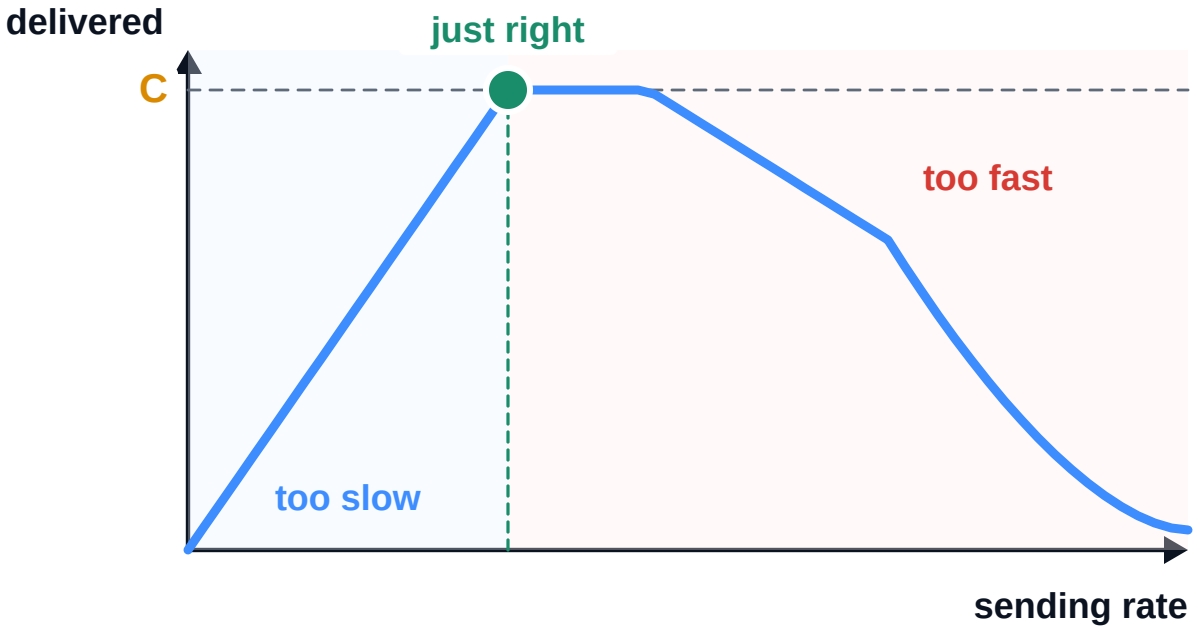
What is congestion control?

05

Control the sending rate

Parts 1 to 4 showed what congestion is, why it happens, why it is hard to control and what it costs. Congestion control is the sender’s answer: keep its sending rate at what the path can carry, no faster and no slower. This part says what that rate is, and gives the sender a way to set it.

Where on Part 4’s curve should a sender be?



Too slow

the link has room to spare:
capacity that nobody uses

Just right: the knee

the sending rate equals C:
the link is full, nothing waits

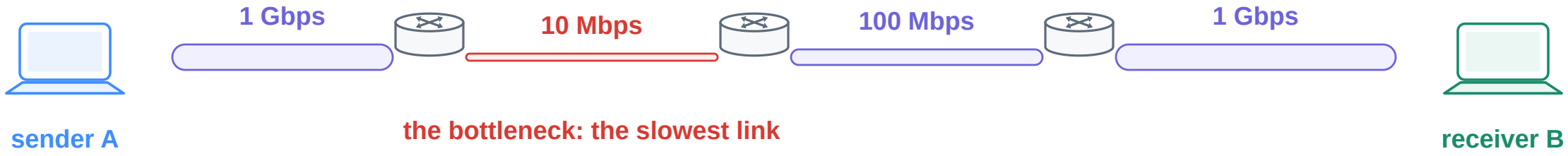
Too fast

the buffer fills, packets drop,
repeats pile up: delivered falls

Part 4’s curve, with one sender: the load it offers is its own sending rate.

At the knee: fast enough to fill the link, no faster. That is congestion control.

What rate fills the path without overflowing it?



Send at 10 Mbps: the bottleneck is exactly full, and nothing waits.

Send faster than 10 Mbps, and Part 4's costs follow, in this order:

packets wait in its buffer → **the delay** grows → **the buffer** overflows: loss → **repeats: what** arrives falls

The bottleneck's rate. Faster than that, packets only wait, then drop.

Two questions every sender has to answer

01



How can a sender control its rate?

what it can change, and how that sets the rate

Part 5 answers it

02



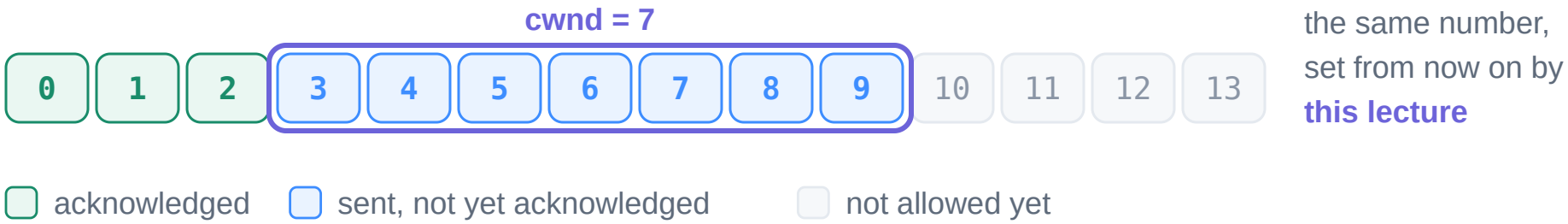
What rate should it aim for?

the bottleneck's, which nothing on the path tells it

Parts 6 and 7 answer it

First the knob, then where to set it.

How can a sender control its sending rate?



How W was set before

a fixed number, chosen once so that the pipe would stay full

it never moved while the connection was running

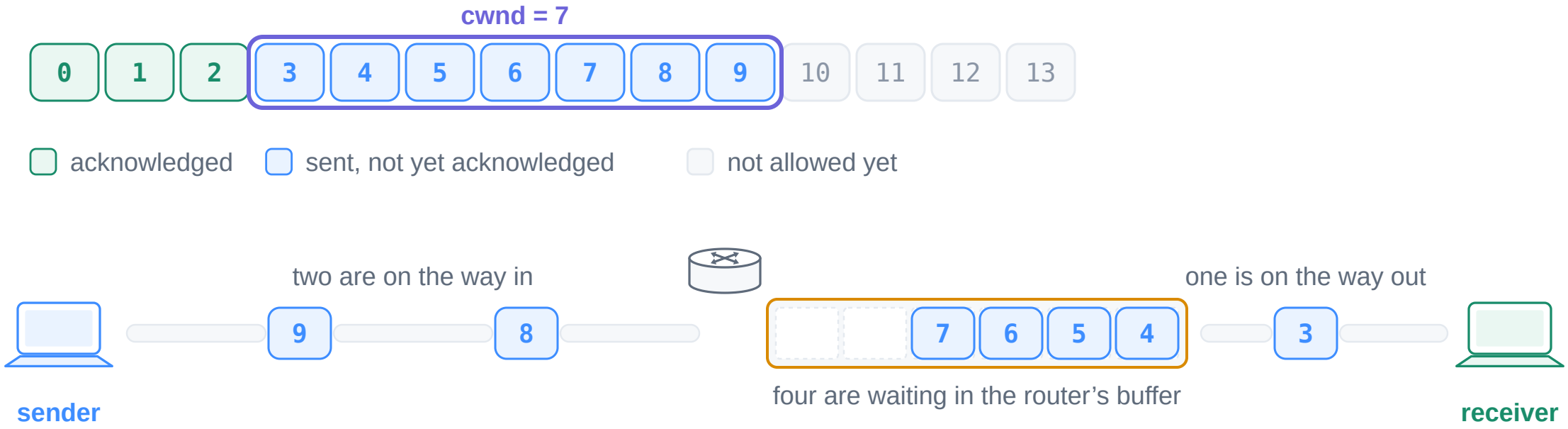
How cwnd is set from here

the sender recomputes it constantly, from what the ACKs tell it

it moves on every ACK and on every loss, for ever

Through its sliding window, now called cwnd. Only how the window is chosen changes.

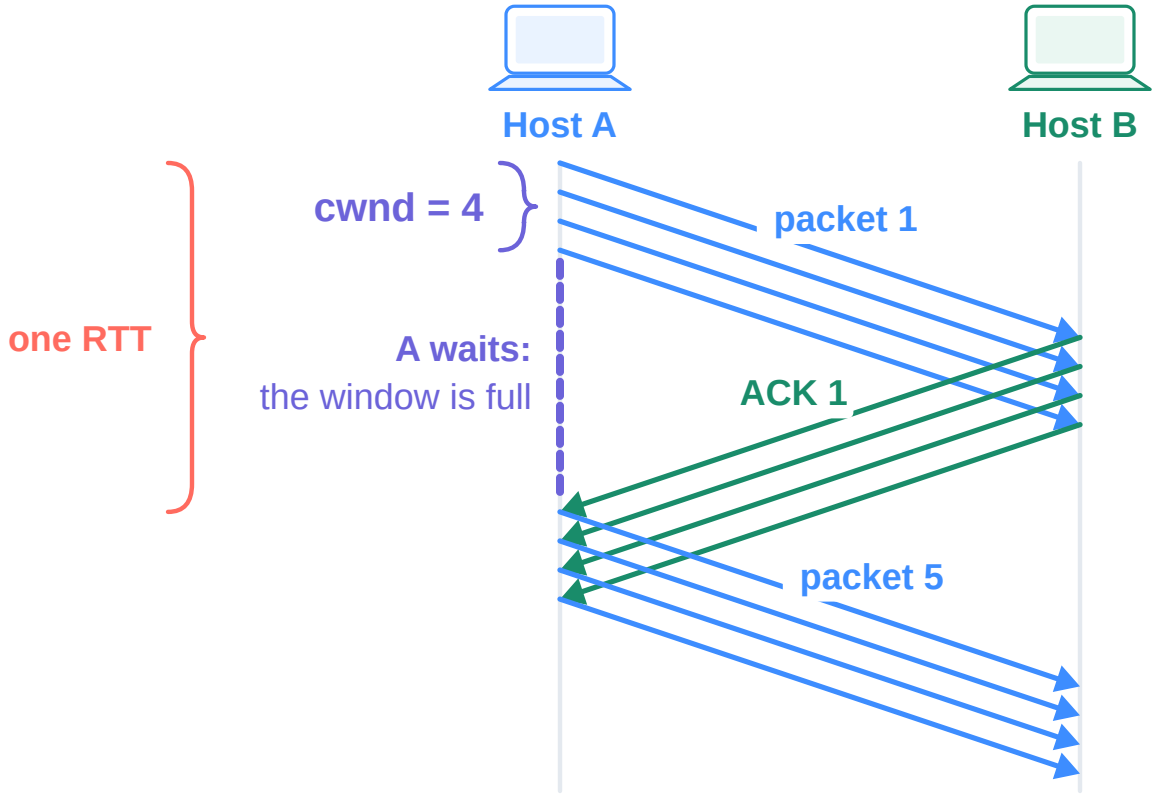
The blue slots are packets that are inside the network



Seven blue slots, seven packets inside the network. cwnd is that count.

A packet sent and not yet acknowledged is a packet inside the network right now.

What does a sender do with cwnd = 4?

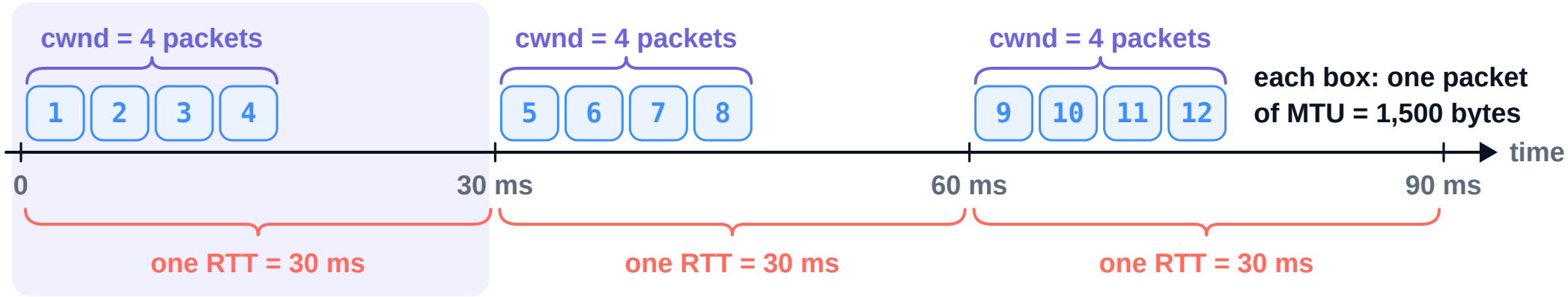


1. A sends **cwnd = 4** packets, one right after another.
2. Four are in the network: the window is full, so **A waits** for an ACK.
3. B sends back an **ACK** for every packet.
4. Each ACK frees one slot, and one new packet goes: 5, 6, 7, 8 leave one **RTT** after 1, 2, 3, 4.

One window leaves every RTT.
The returning ACKs set the pace: the ACK clock.

The window refills only as ACKs return, so one window of cwnd packets leaves per RTT.

Why is the rate cwnd × MTU / RTT?



In one RTT, A sends one window: cwnd packets.

Each packet is **MTU** bytes, so one RTT carries cwnd × **MTU** bytes.

rate = bytes per RTT ÷ RTT = cwnd × **MTU** / RTT

= 4 × 1,500 B / 0.03 s = 200,000 B/s, × 8 bits per byte = 1.6 Mbps

Every RTT one window leaves: cwnd packets of MTU bytes. So rate = cwnd × MTU / RTT.

Which part of that rate can the sender change?

$$\text{rate} = \text{cwnd} \times \text{MTU} / \text{RTT}$$

✗ MTU

the path picks it

one packet is as big as every link on the way will carry: 1,500 bytes on Ethernet

✗ RTT

distance and queueing pick it

a sender can measure RTT, but it cannot make the path shorter

✓ cwnd

the sender picks it

a number the sender keeps in its own memory, and is free to change at any time

“What rate should I aim for?” is now “what should cwnd be?”

Nothing on the path says. Part 6 is how the sender finds out.

Two of the three terms belong to the path. The third is the sender’s to set.

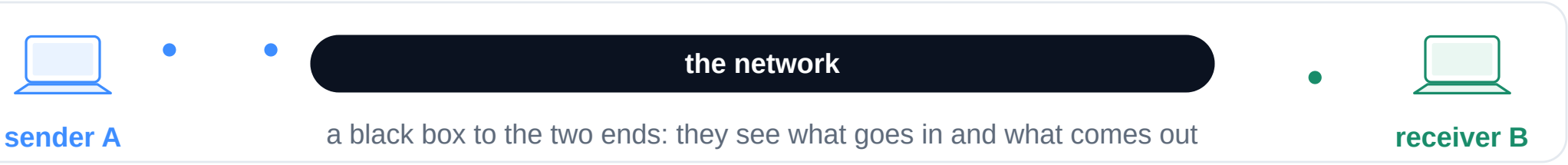
How can a sender find a rate that nothing tells it?

06

Probing for the rate

The sender can set cwnd, but it cannot see the bottleneck. The network could warn it; on the internet it mostly does not. So the sender tries a little faster, and reads what comes back.

Who tells a sender that it is sending too fast?



1 · The network tells the ends



A router sees its own queue grow and warns the ends: “too fast”.

used mainly inside data centers (ECN)

2 · The two ends work it out



The network says nothing. The sender reads the ACKs that come back and sets its rate.

used on most of today’s internet

Either the network says “too fast”, or the two ends work it out between them.

How does ECN let a router say “too fast”?



1 A’s packets say “ECN-capable”
two bits in the IP header tell the routers that this sender understands a mark

2 The router marks instead of dropping
its queue is building, so it sets the two bits to CE: “congestion experienced”

3 B echoes the mark
B sees CE and sets the ECE flag in its ACKs, so the mark travels back to A

4 A slows down, and nothing was lost
A cuts cwnd as it would after a loss, and sets CWR to tell B it has reacted

Slide 15’s router had no field to say “slow down”: ECN (RFC 3168, 2001) adds two bits to the IP header, and two flags for the echo. The lecture after next returns to it.

ECN tells the sender while the queue is still building, before any packet is lost.

Why is ECN common in data centers, rare on the internet?

	A data center	The internet
Who runs it	one company owns every switch and every server	tens of thousands of networks, each run by its own operator
How big	one site: any two servers are only a few switches apart	billions of hosts; one path can cross many networks
To use ECN	turn marking on in every switch, and run the same rule (DCTCP) everywhere	the busy router must mark, and no network on the path may clear the bits
So	**the network tells the ends: way 1**	**the two ends work it out: way 2**

DCTCP's standard limits it to networks "under a single administrative domain" (RFC 8257). On the internet, L4S (2023) builds on ECN, and some ISPs began deploying it in 2025.

One owner can turn ECN on everywhere. The internet has no single owner.

The rest of this lecture: the two ends work it out

1 · The network tells the ends



A router sees its own queue grow and warns the ends: “too fast”.

inside data centers (ECN)

2 · The two ends work it out



The network says nothing. The sender reads the ACKs that come back and sets its rate.

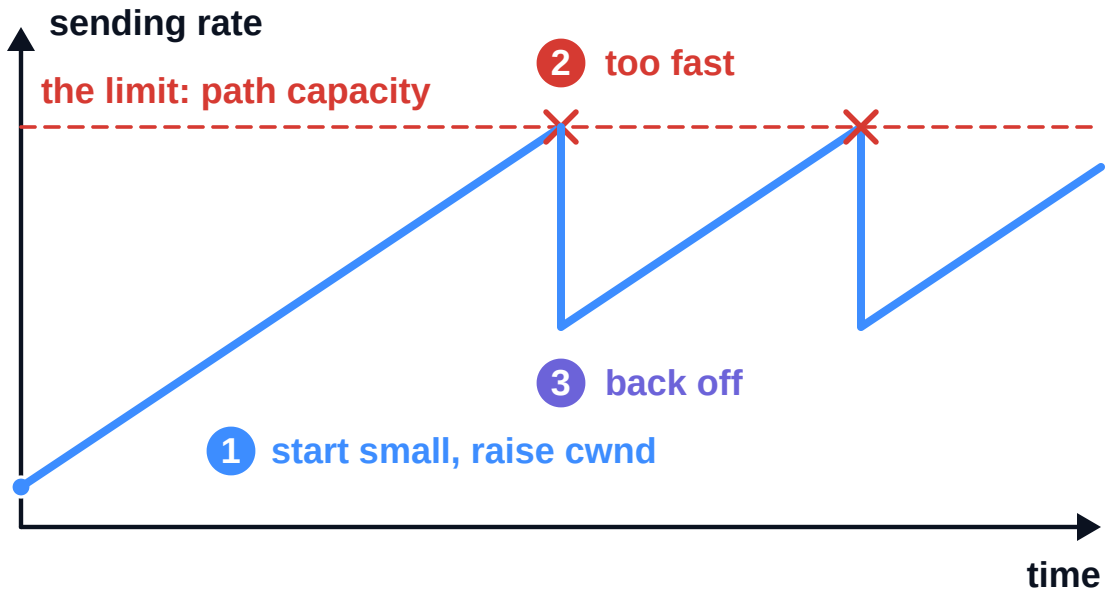
the rest of this lecture

So how does a sender know how fast to send?

Nothing on the path tells it: not the rate, not the limit, not who else is sending.

The network stays silent, so the sender has to find its rate on its own.

How does a sender find a limit it cannot see?



1 Start small, raise cwnd

each step up sends a little faster than the one before

2 Until a signal: too fast

the sender cannot see the limit, but it can tell when it crossed it

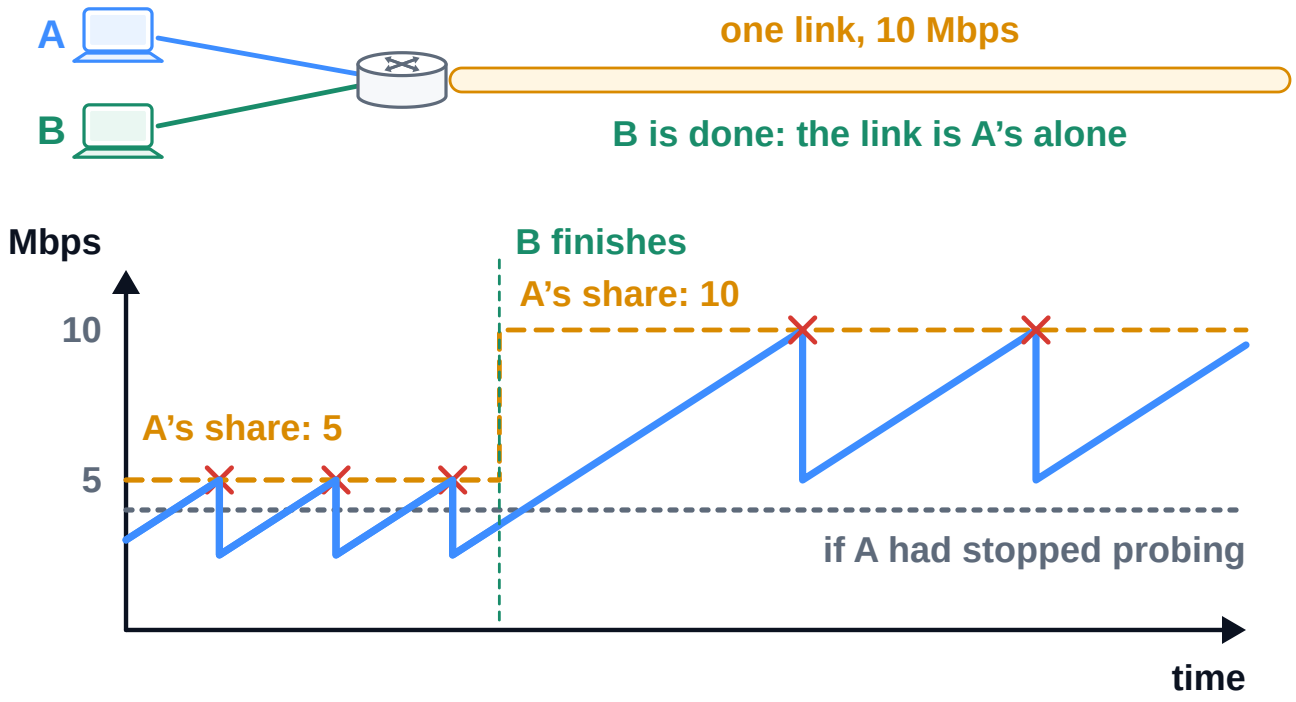
3 Back off, then climb again

drop well below the limit, then raise cwnd slowly once more

The limit is the path's capacity: the rate of its slowest link, the bottleneck (Part 5), or this sender's share of that link when other flows use it too.

Climb until a signal says too fast, back off, and climb again.

Once it has found the limit, why keep probing?



The limit moves

B's download finishes, so A's share of the link doubles: 5 → 10 Mbps

A sender that stops probing

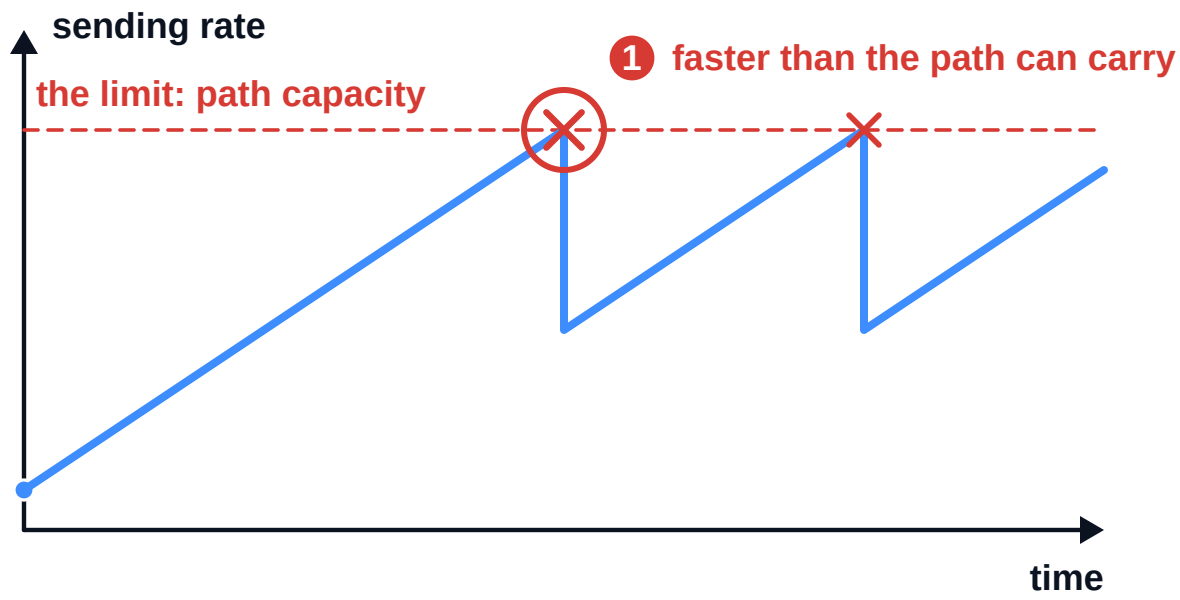
stays at 4 Mbps, so once B is gone 6 of the link's 10 Mbps sit idle

So probing never stops

the sawtooth is the steady state: it follows a limit that moves

Because the limit moves. Only a sender that keeps probing can follow it.

How can a sender tell that it is now too fast?



1 Probing crosses the limit
at the mark, A is sending faster than the path can carry

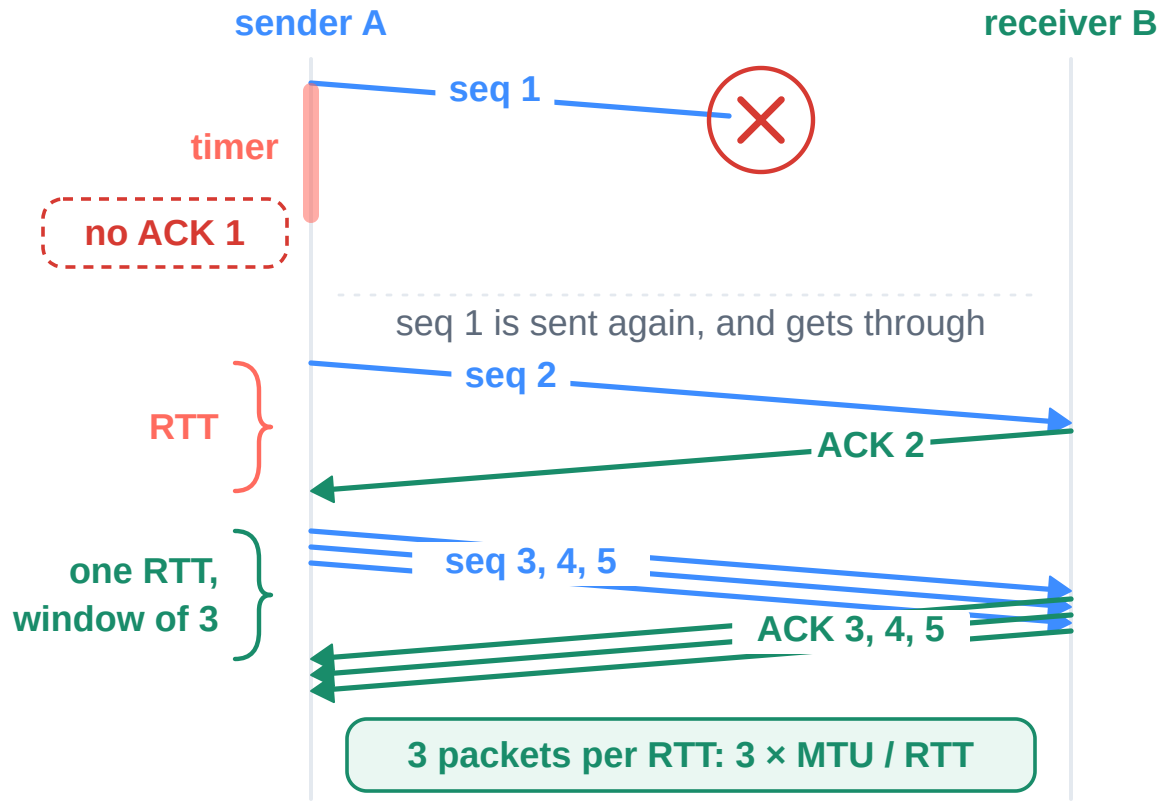
2 How does A know that?
nobody tells it: on the internet the network stays silent (way 2)

3 It has to infer it
from the only thing it has: its own packets and the ACKs that come back

4 What can A measure?
from its own packets and the ACKs, about the network between them

The only clues are the sender's own packets and the ACKs that come back.

What can a sender measure from the ACKs that come back?



LOSS

an ACK that never came back

DELAY

how long the ACK took: the RTT

THROUGHPUT

data rate: window ÷ RTT

Loss, delay and throughput: three measurements, and no message from the network.

Which of the three should the sender use as its signal?

Classic congestion control uses loss.

A lost packet means a buffer on the path was full: the sender went too fast.

It was already there

every sender had to detect loss anyway, in order to retransmit

It needs no new field

no header bit, no new message, no change to any router

And it is usually right

wired links rarely damage packets, so a loss is almost always a full buffer

Classic congestion control reads a loss as “too fast”: a buffer on the path was full.

What about delay and throughput?

LOSS

an ACK that never came back

loss-based: the classic rule of 1988, and CUBIC, the default in Linux, Windows and Apple stacks today

this lecture

DELAY

a longer RTT: a queue is building

delay-based: TCP Vegas (1994) slows down when the RTT grows, before any packet is lost

not covered here

THROUGHPUT

how fast the ACKs come back

rate-based: BBR (2016) measures the delivery rate and the shortest RTT, and sends at that rate

not covered here

Delay and throughput carry information too, and many newer algorithms use them.

This lecture builds the classic loss-based rule, and stays with it: the other families are not covered.

Newer algorithms read delay and throughput too. This lecture stays with loss.

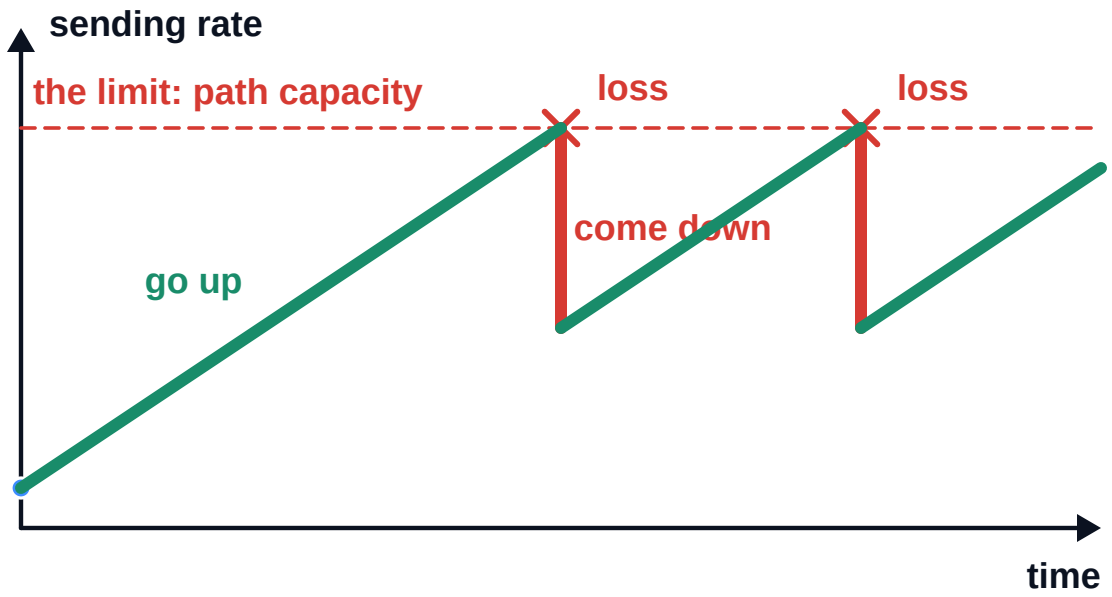
With loss as the signal, how should the rate change?

07

Rate control

Part 6 left the sender one signal: a loss means it went too fast. This part is the rule that uses it: two ways up, two ways down, four rules, and only one, AIMD, is fair.

Loss says “too fast”. What does the sender do with it?



⚠ A loss: the path was full

a buffer overflowed: A was faster than the path could carry (Part 6), so A has to send less

✓ No loss: the path kept up

every packet got through: nothing says A is too fast. Its only clue is to try a little more

? Two questions, then

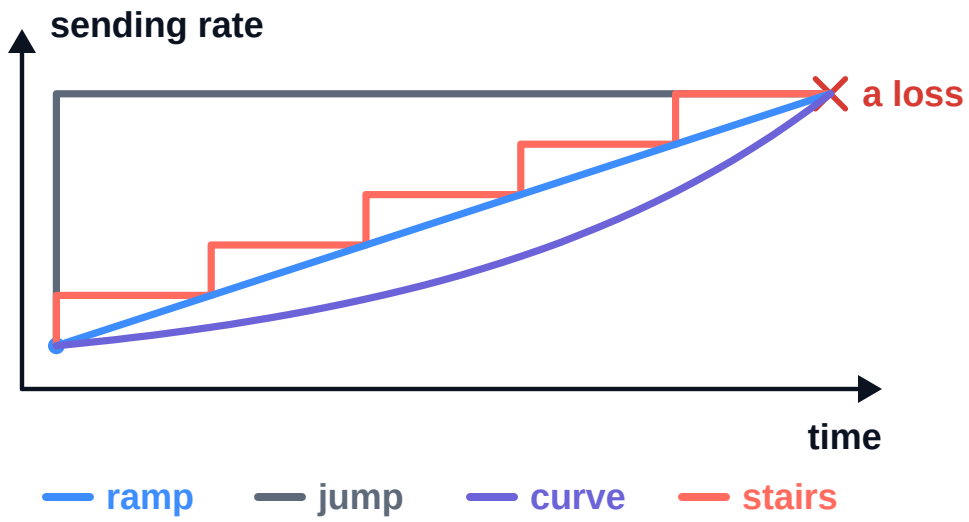
how should the rate go up while nothing is lost, and how far should it come down on a loss?

Come down on a loss, go up while nothing is lost. How much each way is the rule.

How could the rate go up, and how could it come down?

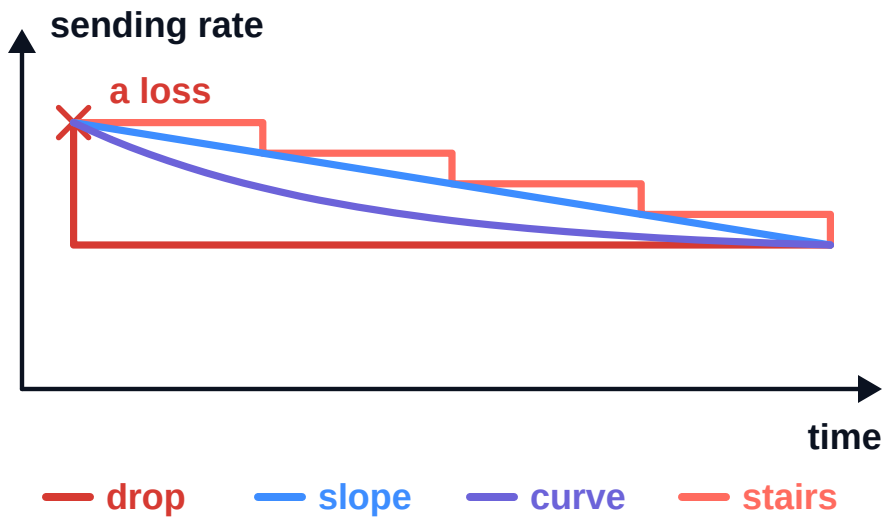
Up, while nothing is lost

how should the rate climb?



Down, on a loss

how far, and how fast, should it come down?



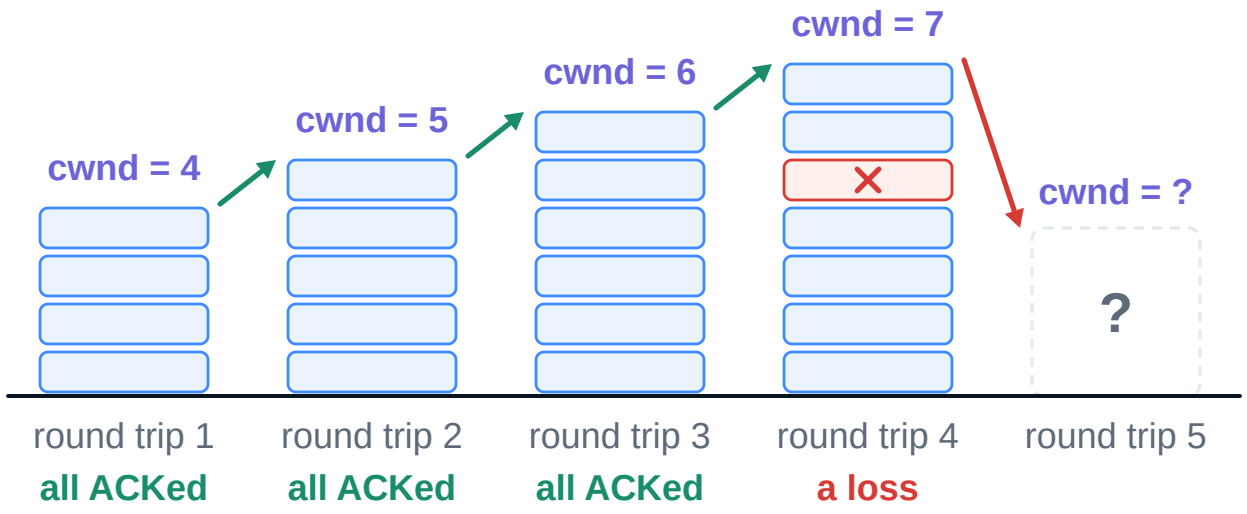
Every shape is one rule: by how much does the rate change per round trip?

Two families cover them: add a fixed amount, or multiply by a fixed factor. Up and down: four rules.

Many shapes, two families of step: a fixed amount or a fixed fraction, up and down.

What makes cwnd go up, and what makes it come down?

each column is one round trip; each box is one packet of its window



✓ Up: every packet ACKed

the path carried the whole window, so the next round trip sends more: one step up per round trip

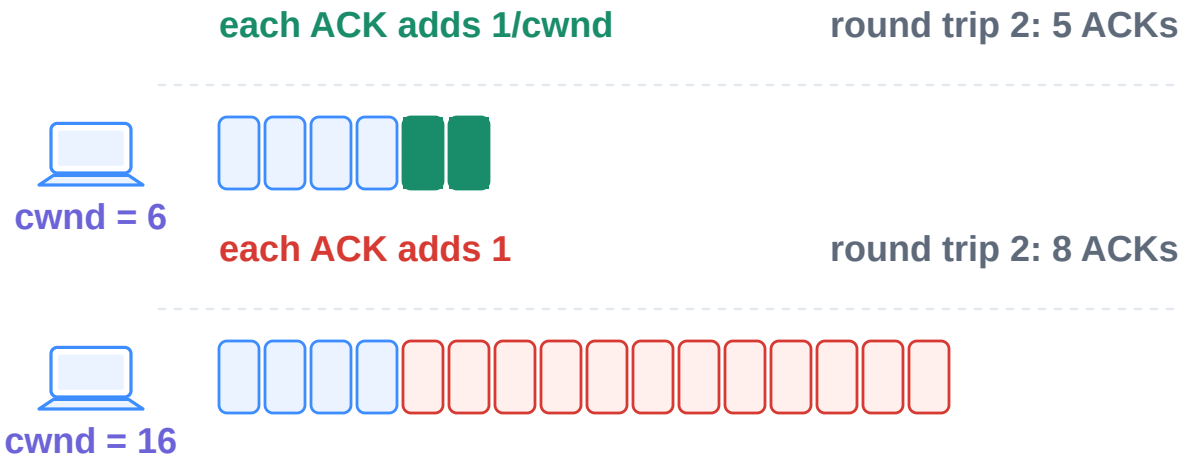
⚠ Down: a packet lost

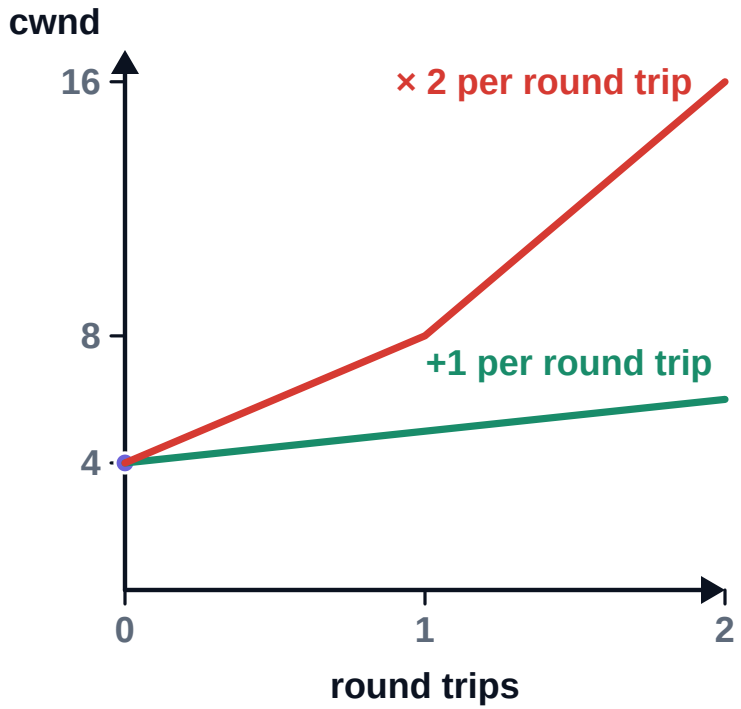
a buffer overflowed, so the next round trip sends less: one step down, however many were lost

But a sender sees ACKs one at a time, not round trips. So each ACK has to move cwnd a little: by how much?

One decision per round trip: every packet ACKed, step up; a loss, step down.

Each ACK moves the window. By how much?





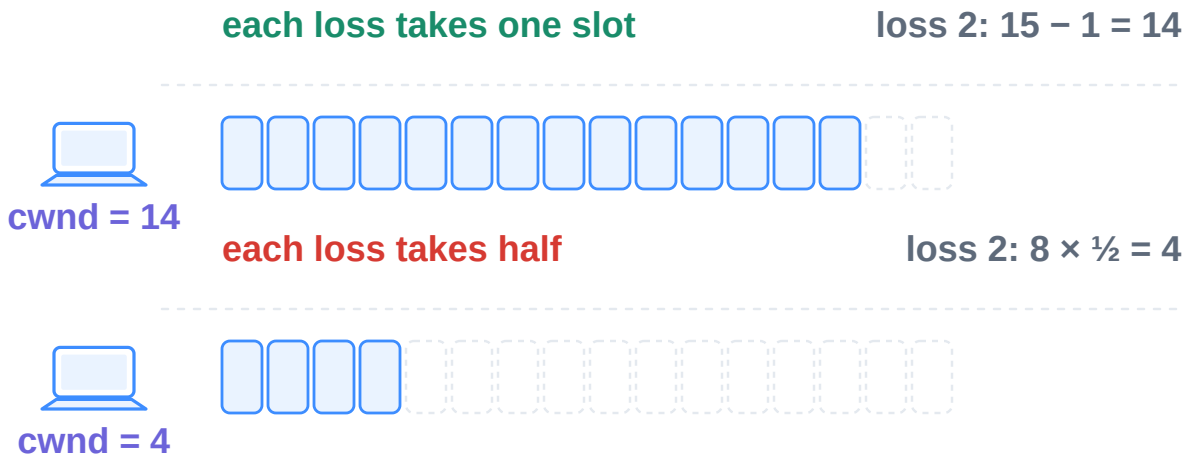
One round trip brings back cwnd ACKs, one per packet.

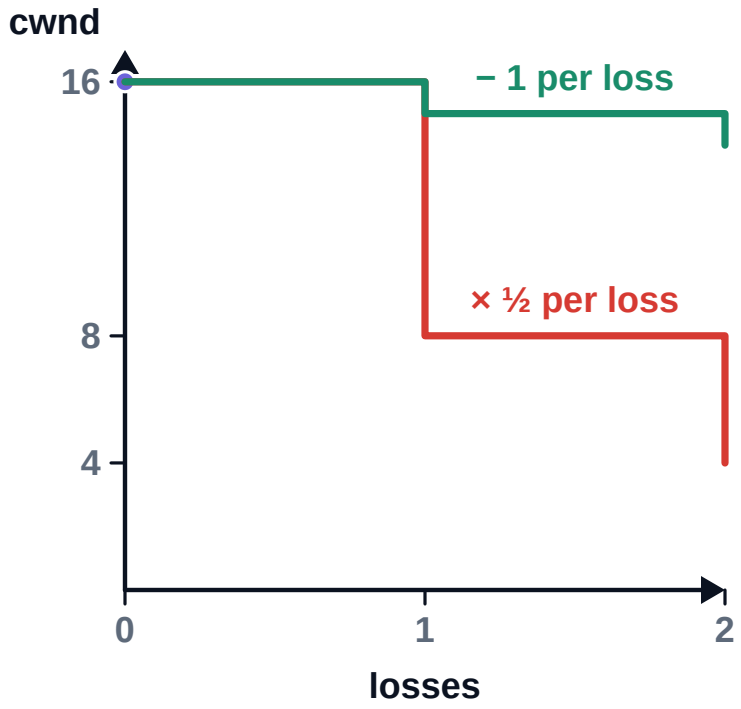
+1/cwnd per ACK: the fractions add up to one packet per round trip, whatever cwnd is. That is additive.

+1 per ACK: cwnd more per round trip, so cwnd doubles. That is multiplicative (Part 8 will use it, at the start).

1/cwnd per ACK: one packet more per round trip. 1 per ACK: twice as many.

A loss moves the window down. By how much?





Take away a fixed amount, or a fixed fraction.

− 1 per loss: 16, 15, 14, the same small step
however big cwnd is. That is additive decrease.

× ½ per loss: 16, 8, 4, half of whatever is there,
so a big window takes a big step. That is multiplicative.

− 1 per loss barely dents a big window. × ½ per loss cuts it in proportion.

How big is a step? Two ways up, two ways down

Going up: by how much?

AI **additive increase**
add a fixed amount: cwnd + a
a = 1 (+1/cwnd per ACK): 4, 5, 6, 7 …

MI **multiplicative increase**
multiply by b > 1: cwnd × b
b = 2 (+1 per ACK): 4, 8, 16, 32 …

Coming down: by how much?

AD **additive decrease**
subtract a fixed amount: cwnd − a
a = 1: 32, 31, 30 …

MD **multiplicative decrease**
multiply by c < 1: cwnd × c
c = ½: 32, 16, 8 …

Additive moves by the same step however big cwnd is; multiplicative moves by the same fraction.
So a multiplicative step is small when cwnd is small, and violent when it is large.

One choice up, one choice down: four possible rules.

Additive is a fixed step, multiplicative a fixed fraction, up and down.

Four possible rules. Which one should a sender use?

up: how much more, each round trip

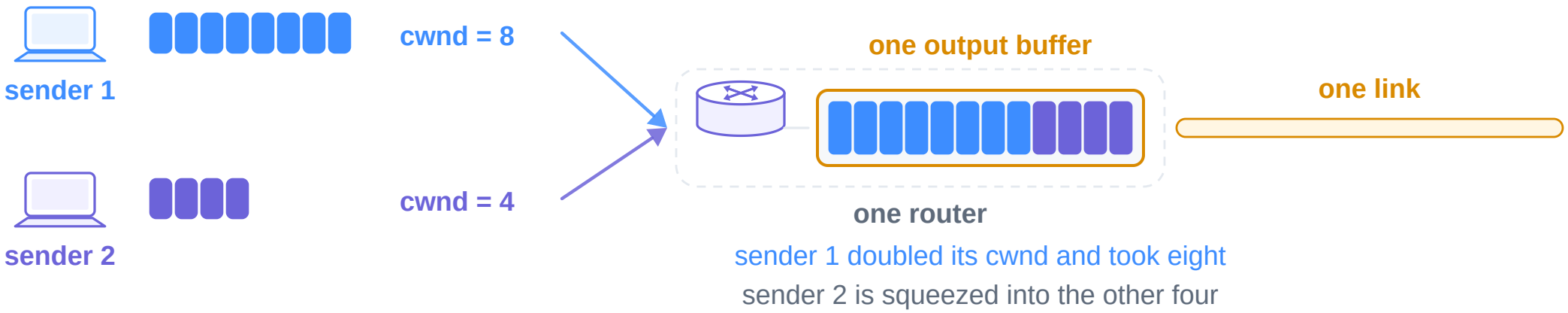
down: how much less, on a loss	AI: + a per round trip	MI: × b per round trip
AD − a per loss	 **AIAD** + a up, − a down	 **MIAD** × b up, − a down
MD × c per loss	 **AIMD** + a up, × c down	 **MIMD** × b up, × c down

Alone on a link, each of the four finds the limit and oscillates around it.

So which one should a sender use?

Two ways up and two ways down make four rules. The next slides pick one.

Two senders, one bottleneck: who gets how much?

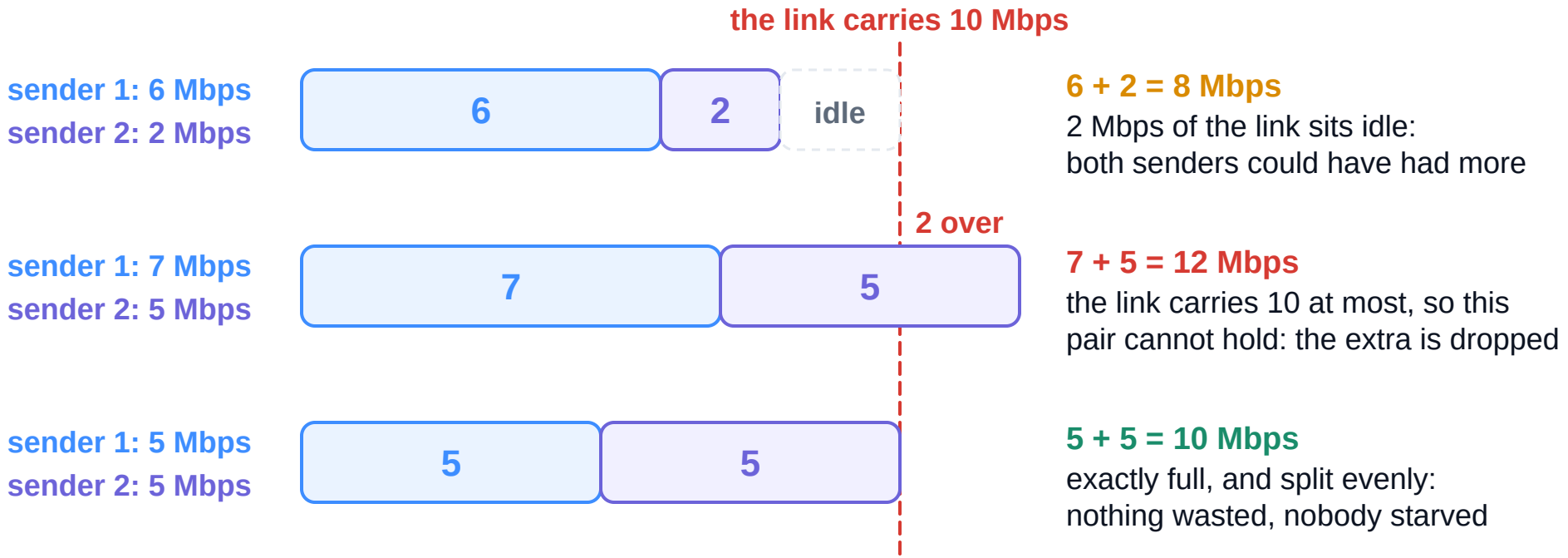


We want the two to get equal shares of the link.

But neither knows the other exists, and they cannot talk: the rule itself has to get them there.

A fair split has to come out of the rule itself: the two senders never coordinate.

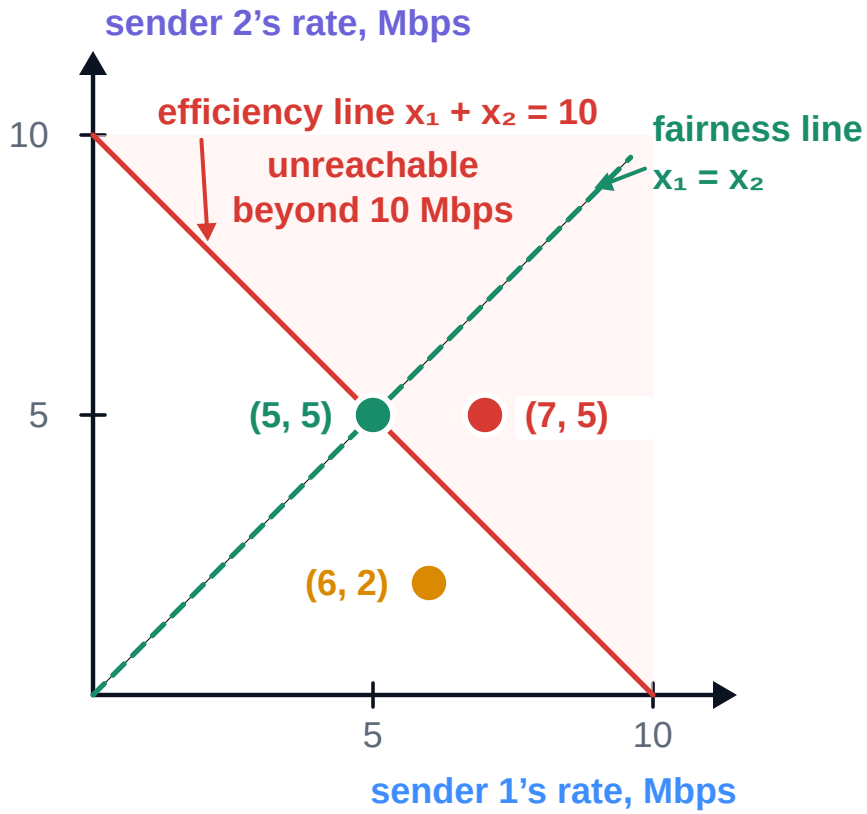
Two senders on one 10 Mbps link: three ways it can go



What each sender is doing right now is just a pair of numbers. Here are three of the pairs.

Too little wastes the link, too much cannot get through, and one pair does neither.

Each of those pairs is one dot on a chart

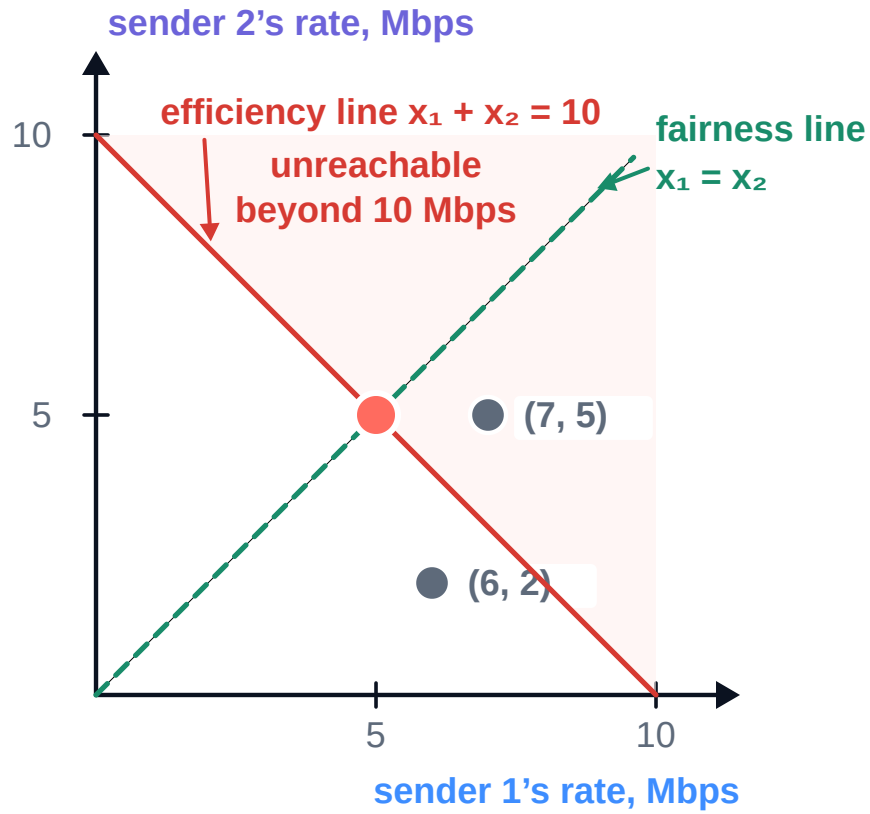


6 | 2	→	(6, 2)	the link is not full
7 | 5	→	(7, 5)	beyond the link's capacity
5 | 5	→	(5, 5)	full, and split evenly

Go right to sender 1's rate, up to sender 2's, and mark the spot.
No time axis here: one dot is one instant, and it moves whenever one of the two rates changes.

The chart is not a graph over time. It is a map of who is sending how much.

Two senders, one link: every possible split



The efficiency line $x_1 + x_2 = 10$

every pair that exactly fills the link
(6, 2) sits below it: capacity wasted
(7, 5) sits above it: it cannot hold

The fairness line $x_1 = x_2$

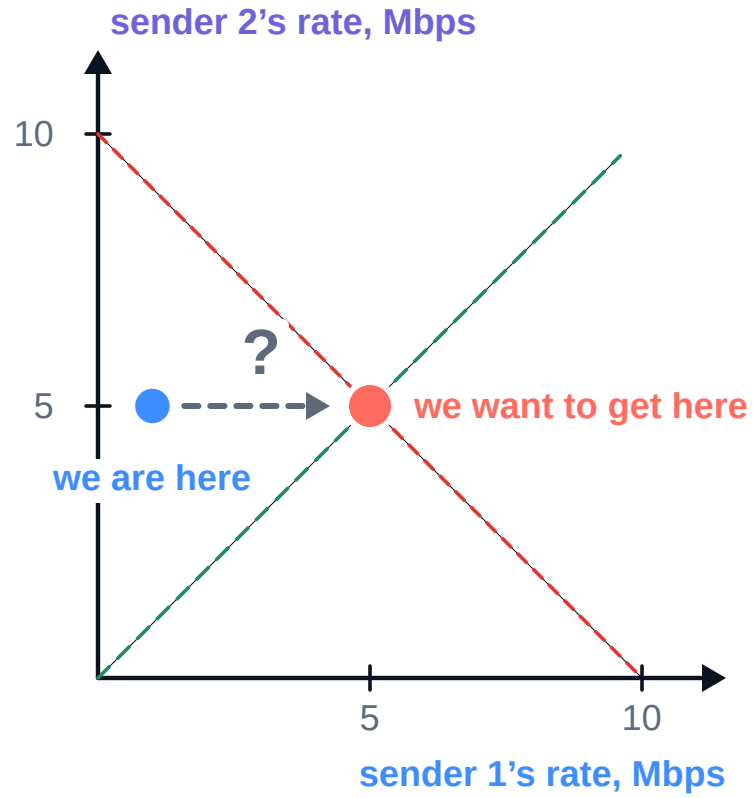
every pair where the two get the same
(7, 5) is off it: sender 1 is taking more
a point on it may still waste the link

Where they cross (5, 5)

full link and equal shares at once
the only pair that is both
a rule has to push any point here

Each point is one way to split the link. A rule is a way of moving that point.

How do the senders get the point to the middle?



The target

(5, 5): the link full and the two shares equal: the one point that is both

What neither sender can see

the other sender's rate, where the two lines are, or that (5, 5) exists at all

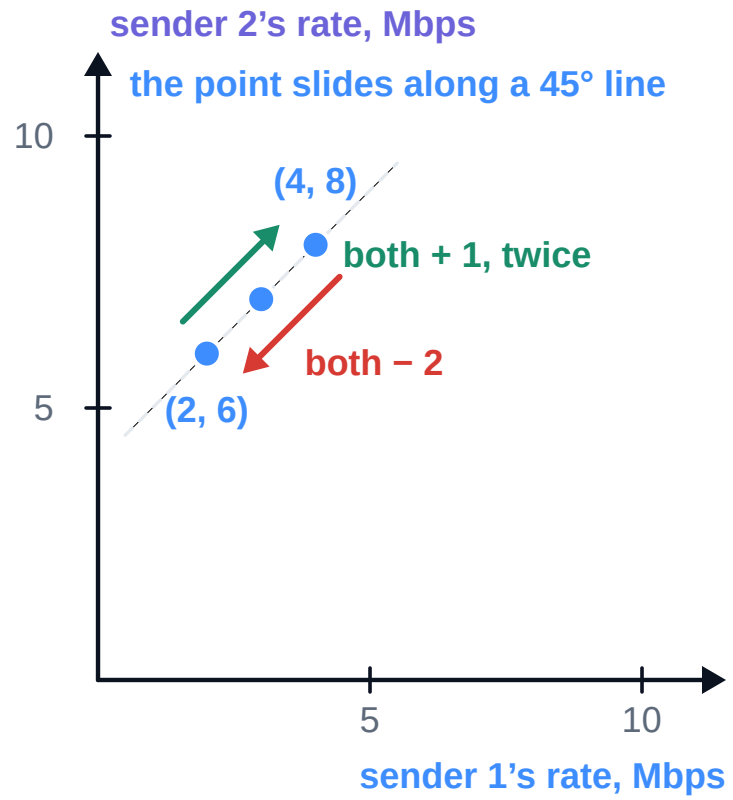
So what is a sender left with?

its own cwnd, and one bit coming back: a loss, or no loss. Both run the same rule.

Both raise cwnd with no loss, lower it on a loss. Which of the four rules brings the point here?

Nobody can steer towards a point they cannot see. The rule has to get there blind.

Add the same amount to both: does the split change?



Both add the same amount

sender 1: 2 → 3 → 4
sender 2: 6 → 7 → 8
the gap stays 8 − 4 = 4

Both subtract the same amount

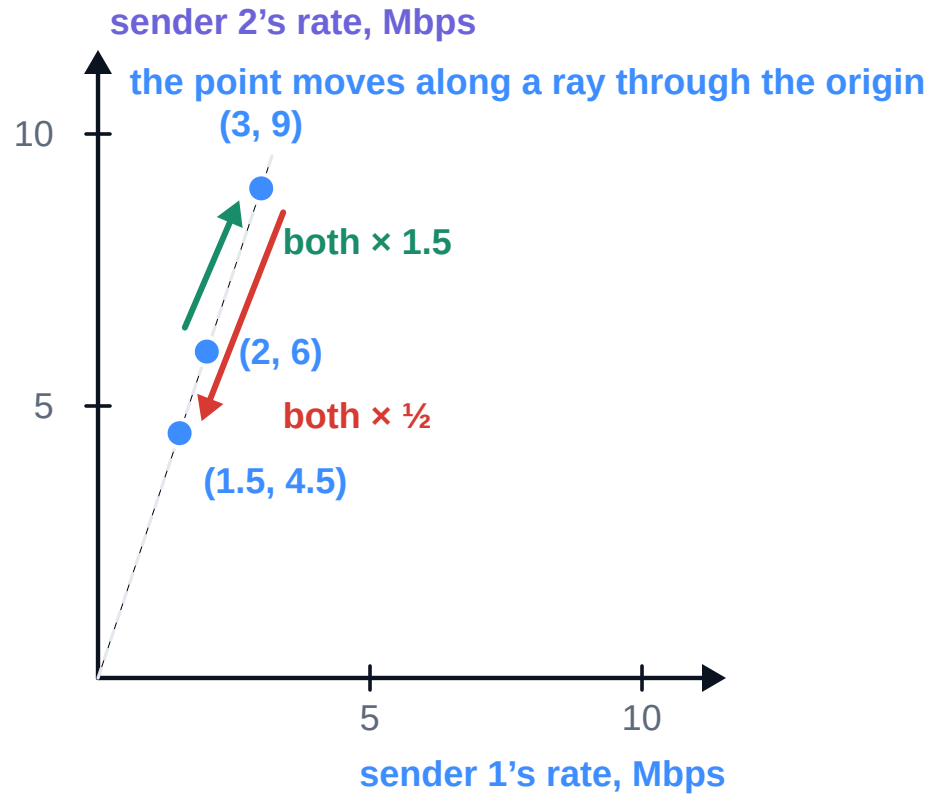
sender 1: 4 → 2
sender 2: 8 → 6
the gap stays 6 − 2 = 4

Why: the gap cannot change

$(x_2 + a) - (x_1 + a) = x_2 - x_1$
the same a is added to both and
cancels in the difference

No. Adding or subtracting the same amount slides the point along 45°: the gap is kept.

Multiply both by the same factor: does the split change?



Both multiply by 1.5

sender 1: 2 → 3, sender 2: 6 → 9
the gap grows: 4 → 6
the ratio stays 3 : 1

Both multiply by ½

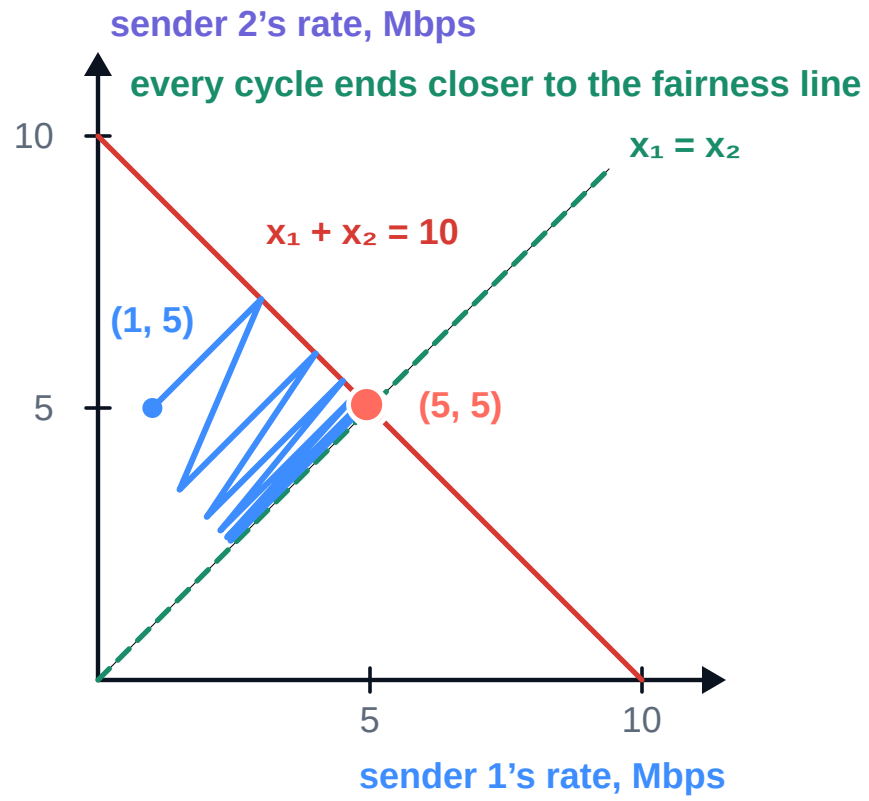
sender 1: 3 → 1.5, sender 2: 9 → 4.5
the gap shrinks: 6 → 3
the ratio stays 3 : 1

Why: the gap scales with c

$c \cdot x_2 - c \cdot x_1 = c \cdot (x_2 - x_1)$
halving both halves the gap;
the ratio $x_2 : x_1$ is untouched

The gap changes in proportion: halving both halves it. The ratio never changes.

Combine them: add on the way up, halve on a loss



+ Up: both add 1 per round trip

along 45°, so the gap is kept,
until the link is full: the red line

↓ A loss: both halve

towards the origin, so the gap
is halved: 4, 2, 1, ½ …

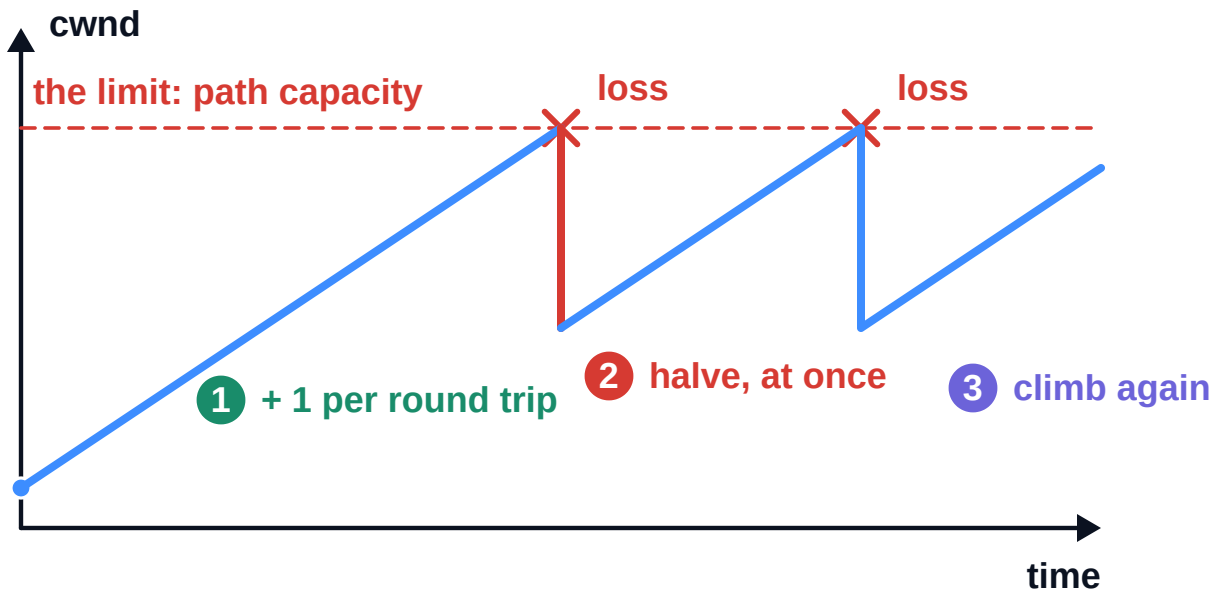
✓ So the split moves to the middle

each climb fills the link again,
each loss halves the gap: it converges

AIAD never changes the gap, MIMD never the ratio.
Both stay off the middle for ever. Only AIMD moves the split.

AIMD converges to an equal share from any starting point. That is the rule to use.

AIMD, as the sender runs it



1 No loss this round trip

cwnd = cwnd + 1
(each ACK adds 1/cwnd: a = 1 packet)

2 A loss

cwnd = cwnd / 2, at once
(half the window: c = ½)

3 Then back to step 1

climb from the halved window,
one packet per round trip, until the next loss

One packet more per round trip; half the window on a loss; then climb again.

Why does the gap shrink, cycle after cycle?

what happens	sender 1	sender 2	the gap
start	1	5	4
grow: + 2 each, until the link is full	3	7	4
loss: halve both	1.5	3.5	2
grow: + 2.5 each	4	6	2
loss: halve both	2	3	1
grow: + 2.5 each	4.5	5.5	1
loss: halve both	2.25	2.75	0.5
grow: + 2.5 each	4.75	5.25	0.5
… and so on	→ 5	→ 5	→ 0

Each grow row is many round trips of + a, until the link is full again: from a total of 5, that is 2.5 each.

Growing cannot change the gap

it adds the same number to both, so the gap comes through unharmed

Halving must change the gap

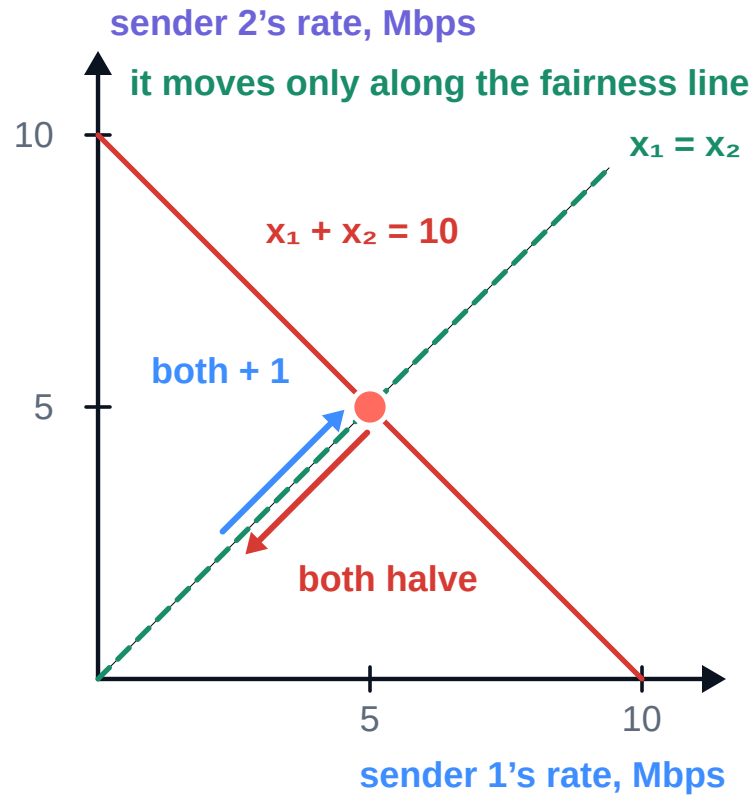
it multiplies both by one half, so the gap is halved as well

So the gap halves every cycle

4, 2, 1, ½, ¼ … → 0, and the increase keeps the point on the full link

The gap survives every increase and is halved by every decrease, so it goes to zero.

AIMD reaches the middle. How does it stay there?



Equal stays equal

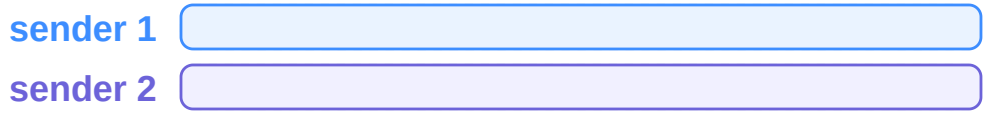
at $x_1 = x_2$ both senders make the same move, so a gap of 0 stays 0

But it never stands still

it cannot see the middle, so both keep adding until a loss, halve, and climb again

Pushed off, it comes back

a loss that only one sender sees opens a gap, and every cycle after that halves it again



2.5 → 5 → 2.5 Mbps, the two always equal

Equal stays equal; the point swings up to the middle and back, once per loss.

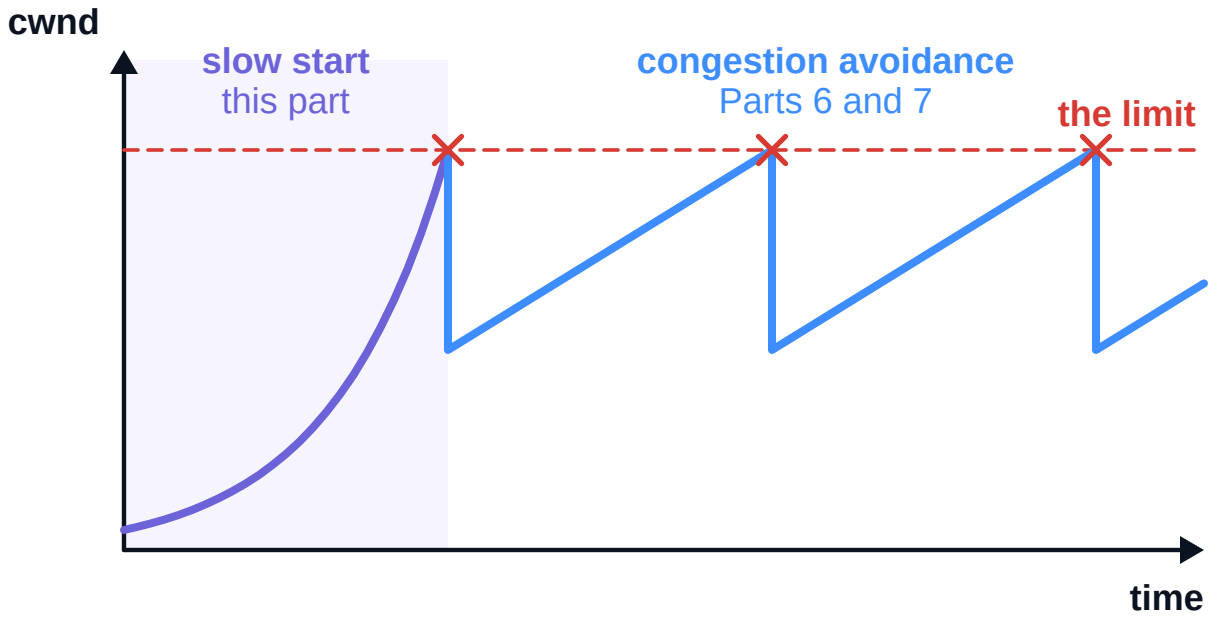
How does a new connection get up to speed?

08

Slow start

Congestion control runs in two phases. Everything so far was the second one, congestion avoidance: AIMD around the limit. This part is the first, slow start: why a connection does not simply begin in congestion avoidance, and what it does instead.

Two phases: slow start, then congestion avoidance



Congestion avoidance: done

around the limit: + 1 per round trip,
halve on a loss, climb again

Slow start: this part

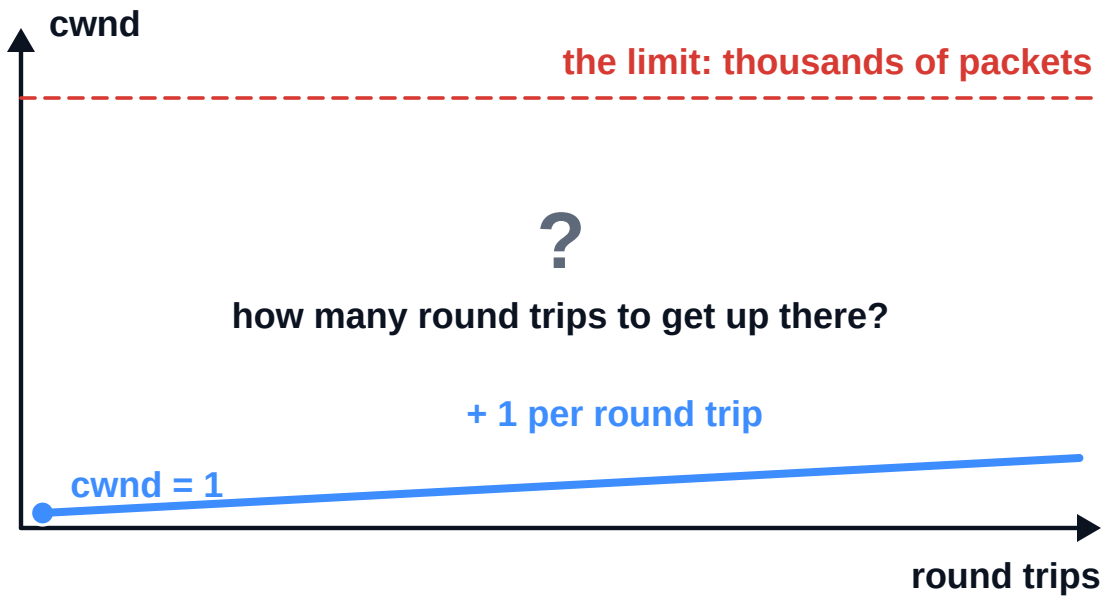
from the first packet up to the limit:
how to get there fast

Why two phases at all?

Why not begin in congestion avoidance?

Parts 6 and 7 were congestion avoidance. Slow start is how a connection gets there.

Why two phases? Why not start in congestion avoidance?



A new connection knows nothing

no idea where the limit is,
so it has to start small

Start in congestion avoidance?

+ 1 per round trip from cwnd = 1:
the next slides count the cost

Or: a faster first phase

get near the limit in a few round trips, then switch to avoidance

From one packet, + 1 per round trip takes for ever. The first phase has to be faster.

How many packets does a 1 Gbps, 30 ms path hold?

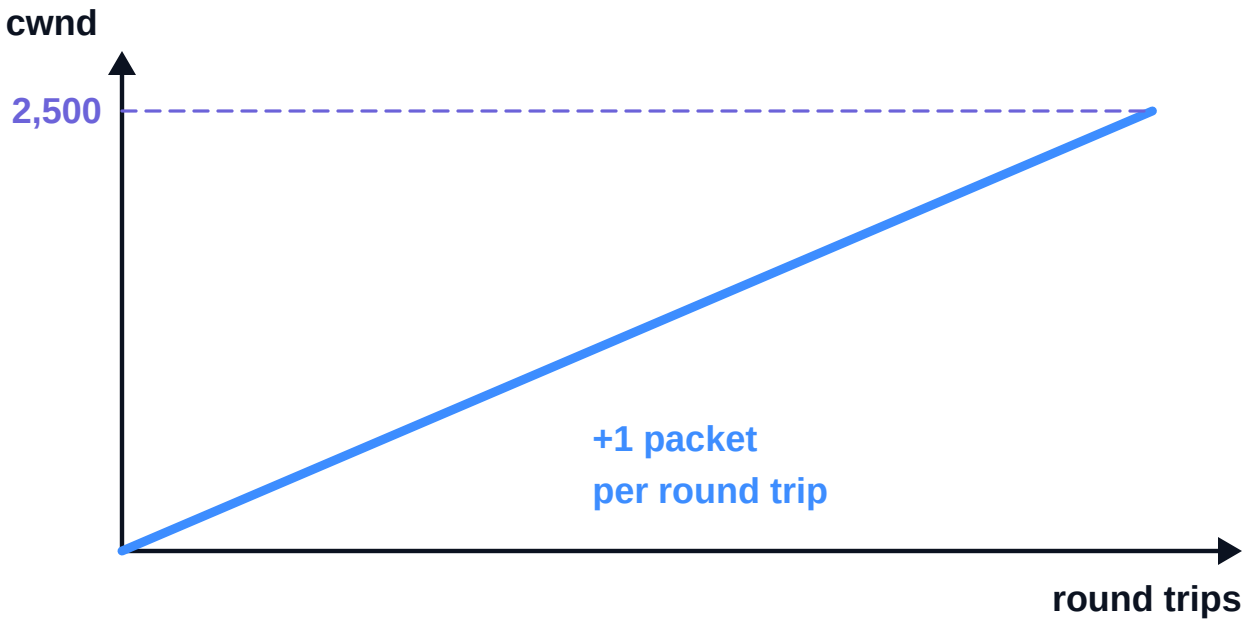
1,500 bytes	1 Gbps	30 ms	cwnd = 2,500
one packet, the MTU	the bottleneck link	the round-trip time	fills the path exactly

cwnd = 2,500	2,500 × 1,500 B / 30 ms = 1 Gbps	the window that keeps the link busy
cwnd = 1	1 × 1,500 B / 30 ms = 0.4 Mbps	where the original TCP started

Every number in this part comes back to these four.

2,500 packets. A connection that starts at one packet has a long way to climb.

How long does additive increase take to reach 2,500?



2,500 round trips

× 30 ms

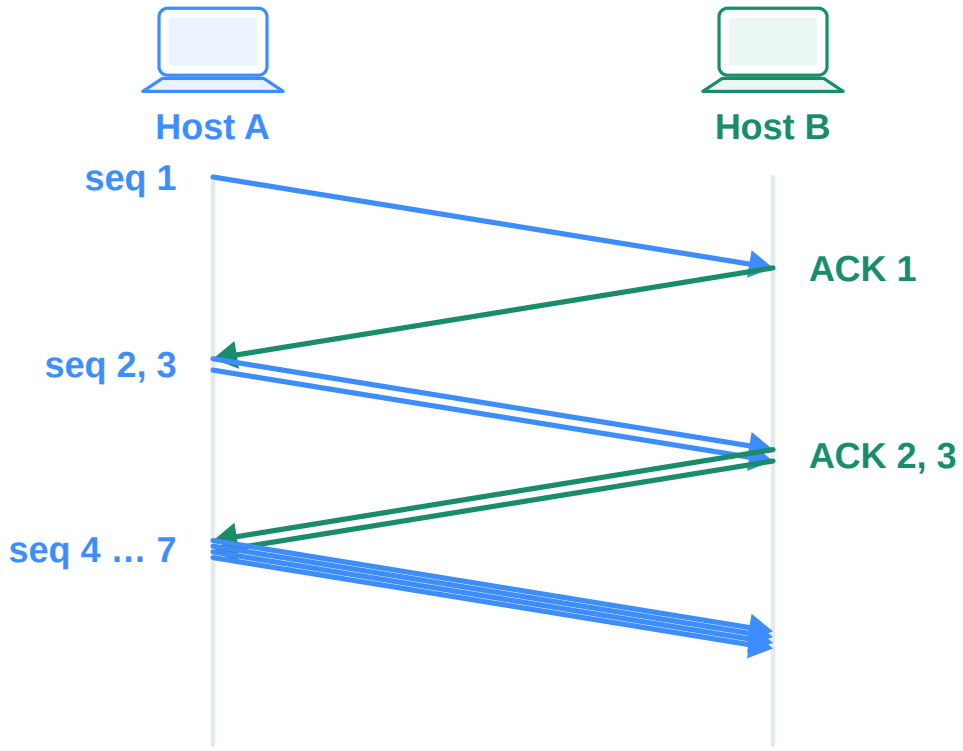
= 75 seconds

A web page is finished in well under a second.

Most connections would never use the link at all.

75 seconds. AIMD is the right rule for steady state and useless for starting.

What if every ACK released two packets?



The rule: one ACK, two packets

an arriving ACK frees the slot it is acknowledging, and the sender puts two packets into it, not one

cwnd += 1 packet per ACK

so a whole window of ACKs

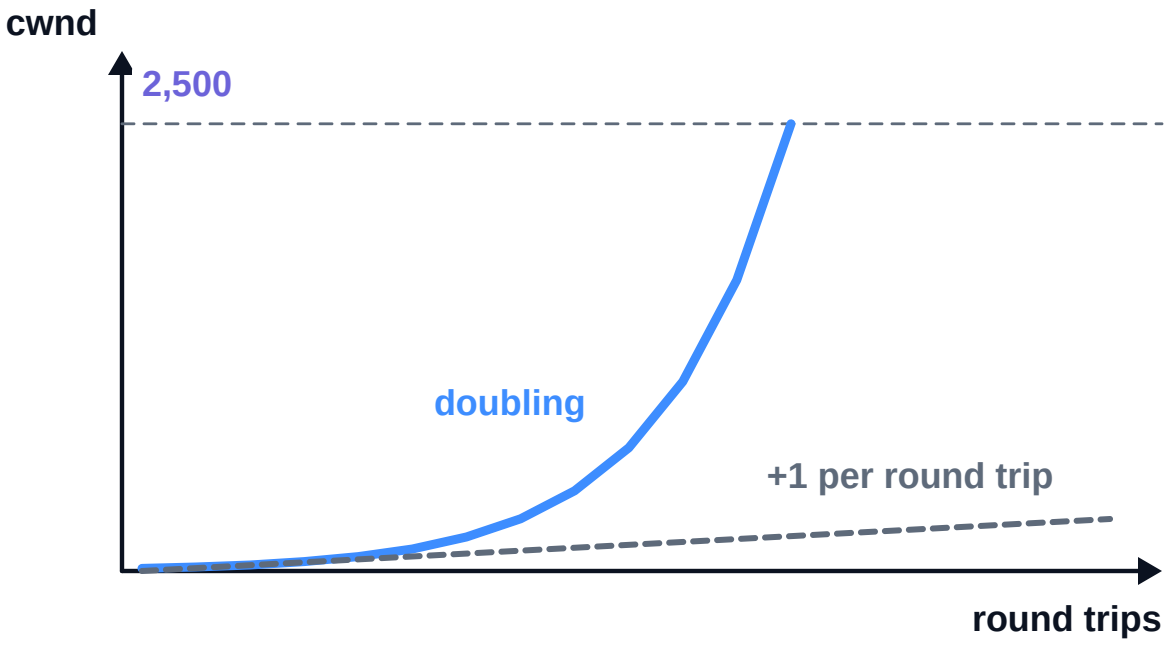
doubles cwnd

`1, 2, 4, 8, 16, …`

one doubling per round trip

Additive increase adds one packet per window. Slow start adds one per ACK.

Doubling: twelve round trips instead of 2,500



additive, +1 each time

2,500 round trips **75 seconds**

doubling

12 round trips **360 ms**

twelve, because $2^{12} = 4{,}096$

the window passes 2,500 on the twelfth doubling, and 12 × 30 ms is 360 ms, well inside the life of a single web request

Doubling turns a 75-second climb into twelve round trips.

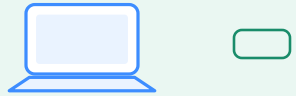
Why is doubling called “slow” start?

Before 1988



a full window of packets
on the link at once,
as soon as the connection opened

After 1988



one packet,
then two, then four:
the link never gets a full window at once

Slow start is slow compared with what it replaced,
not compared with additive increase, which it beats by a factor of two hundred.

The name is historical: it is slower than the burst it was invented to stop.

Why does a connection now start at ten packets?

IW = 10 packets, and at most 14,600 bytes

RFC 6928 writes it IW = min(10 · MSS, max(2 · MSS, 14600)), in TCP payload bytes.

A new connection may put ten packets on the link before it hears anything back.

cwnd = 1	**12 round trips**	**360 ms**
cwnd = 10	**8 round trips**	**240 ms**

Four round trips saved here ($\log_2 10 \approx 3.3$ in general), and a web page is a few round trips long.

Starting at one packet costs a short connection most of its lifetime.

Eight parts, one line each

01	Congestion	More in than out, for long enough that the queue does not drain.
02	Causes	One sender too fast for its path, or many senders together.
03	Challenges	A mesh of shared links, no coordination, and a right rate that keeps moving.
04	The costs	Past capacity, load turns into waiting, repeats and wasted hops.
05	The rate	Aim for the bottleneck’s rate; the lever is cwnd: rate = cwnd × MTU / RTT.
06	Probing	On the internet the ends work it out: climb until a loss, back off, climb again.
07	Rate control	AIMD: add one when nothing is lost, halve on a loss; senders end up equal.
08	Slow start	Start at ten packets, double every round trip until the first loss, then AIMD.

Each part answered one question. Together they make up classic TCP congestion control.

1 Congestion | 2 Causes | 3 Challenges | 4 The costs | 5 The rate | 6 Probing | 7 Rate control | 8 Slow start

One more row in the table

the defect	the mechanism it forces
Bits flip inside a packet	**Checksum**
A cannot see what arrived at B	**ACK from the receiver**
The feedback itself is damaged	**Sequence number**
A packet is lost	**Timer; resend when it expires**
Loss looks the same as delay	**Adaptive timeout, doubled after expiry**
The link sits idle	**Window of bandwidth × RTT packets**
One packet in a window is lost	**Go-Back-N or selective repeat**
A number is reused too soon	**Window ≤ half the sequence space**
Loss is detected late	**Duplicate ACKs, selective ACK**
The receiver overflows	not covered in this course
The network between them overflows	**Congestion window, AIMD**

Row 10 is a real defect with a real mechanism, the advertised window. This course does not spend a lecture on it.

Every mechanism here exists because the protocol before it broke in one way.

Three assumptions this design makes



A loss means congestion

what about a wireless link that loses packets with an empty buffer?



Halving is right

what happens on a 10 Gbps path, where the window is 83,333 packets?



A full buffer is acceptable

what does filling every buffer before reacting do to everyone else's delay?

Each of the three is a design decision, not a law of nature.

The next two lectures replace each of them, and many of today's senders already do.

AIMD is one point in a design space. The next lecture walks to the other points.